

ACKNOWLEDGEMENT

It gives us immense pleasure to express our deep sense of gratitude and sincere appreciation to our project supervisor, Dr. Souradip De, from the Electrical Engineering Department at MNNIT Allahabad. His constant encouragement, insightful guidance, and unwavering support have been invaluable throughout this journey. His patience in answering our questions, his thoughtful suggestions, and his clear direction have significantly enhanced our understanding and enriched our work. Working under his guidance was not only a learning experience but also a source of genuine inspiration.

We would also like to convey our heartfelt thanks to Prof. Richa Negi, Head of the Electrical Engineering Department, MNNIT Allahabad, for providing the resources and helpful advice that enabled us to complete this work smoothly.

Our sincere gratitude also goes to the entire Department of Electrical Engineering at MNNIT Allahabad for giving us this wonderful opportunity, which has greatly expanded our knowledge and experience.

Finally, we bow our heads in deep reverence and gratitude to Almighty God, whose blessings and grace gave us the strength, wisdom, and perseverance to see this work through to the end.

CHAPTER 1

LITERATURE SURVEY

The rapid growth of unmanned air vehicles (UAVs) has revolutionized industries from agriculture to disaster management. This progress is the development of central strong autonomous flight control systems, which integrates advanced algorithms, sensor fusion and energy-skilled strategies to enable accurate navigation, barrier avoidance and mission execution. This literature survey synthesizes significant progress in important domains including localization, control theory, herd coordination and security, which provides a comprehensive observation of state -of -the -art Research.

The pursuit of autonomous drone's hinges on solving one core challenge: enabling them to navigate spaces where GPS signals vanish, like indoor warehouses or disaster-stricken areas. Back in 2015, a breakthrough came from Mur-Artal and colleagues, who built ORB-SLAM—a camera-based navigation system that mimics how humans perceive depth. By identifying distinct visual markers (called ORB features) in real time, their system constructed 3D maps accurate to a few millimeters, even in flickering lighting. This allowed drones to chart collapsed buildings or dimly lit tunnels, a leap critical for rescue missions [1]. Around the same period, Krajník's team took a different approach, blending KLT feature tracking—a method borrowed from video motion analysis—with Kalman filters. This combo let drones tail moving objects through cityscapes, keeping positioning errors under half a meter, useful for tasks like monitoring traffic [6]. Energy efficiency soon emerged as another hurdle. Graule and collaborators tackled this in 2016 by borrowing ideas from geckos: their drones used electrostatic grips to cling to walls, slashing energy use by 92% during surveillance—like a bird perching to rest [8]. Fast-forward to 2023, and Kaufmann's group pushed speed limits, training drones via reinforcement learning to race through courses at near-human pilot speeds, proving AI could handle split-second navigation decisions [9].

Stability in unpredictable conditions remains a linchpin for drone reliability. Take wind resistance: Alexis's 2017 work introduced controllers inspired by aircraft maneuvering, using "sliding mode" adaptive techniques to keep quadcopters steady in 35 mph winds, hitting 85% accuracy in following preplanned paths [5]. When motors fail mid-flight, redundancy becomes key. Zhang's 2021 solution layered PID controllers

with iterative learning—a method that lets drones "practice" recovering from simulated failures, much like pilots training in simulators [15]. Hybrid drones, which switch between helicopter-like hovering and plane-like gliding, bring their own quirks. Rathi's 2020 CFD models revealed how air vortices disrupt transitions, leading to tilt-rotor designs that shudder less during mode shifts—akin to smoothing a car's gear change [11]. For swarms, Saska's 2016 Model Predictive Control (MPC) framework became a game-changer, choreographing 50+ drones to dodge obstacles in sync, much like flocking birds avoiding trees [4]. These innovations highlight a shift from rigid control logic to adaptable, self-correcting systems.

Extending flight time isn't just about bigger batteries—it's about smarter routing. Yang's 2019 algorithm treated wind as a dynamic map, rerouting drones like sailboats tacking against currents, boosting flight durations by over a quarter [3]. In farming, Vega's team fused multispectral cameras with AI in 2018, teaching drones to spot thirsty crops and optimize watering schedules, lifting yield forecasts by 19% while conserving power [12]. Communication lag can be just as crippling as low battery. Park's 2021 TDMA protocol—a traffic light system for data packets—cut drone fleet delays to 12 milliseconds, vital for missions where a stalled video feed could mean missing a survivor [10]. Meanwhile, Mozaffari's 2019 RL-driven base stations acted like aerial cell towers, learning to hover where signals matter most, achieving near-perfect data delivery in remote regions [7]. Together, these strides show how energy savings stem from both hardware ingenuity and algorithmic wit.

With drones proliferating, security gaps have sparked urgent fixes. Alladi's 2022 blockchain fix for GPS spoofing works like a digital notary, cross-checking location data across drones to catch fakes with 99.3% precision—a must in cities where skyscrapers bounce signals [13]. NASA's UTM system, detailed by Kopardekar in 2020, acts as an air traffic control for drones, using geofencing to ban flights near airports and algorithms to prevent mid-air collisions, scaling to manage thousands per square mile [14]. Yet even vision systems aren't immune: Loquercio's 2018 study showed how subtly altered patterns could fool drone cameras, spurring tougher neural networks that spot obstacles 92% of the time despite such tricks [2]. These layers of defense—encryption, traffic rules, and hardened AI—pave the way for safer urban drone highways.

Drones are now infiltrating niches once deemed impractical. Medical delivery prototypes, drawing from Rathi's hybrid designs, are testing VTOL flights to remote clinics, balancing speed and energy use [11].

Environmentalists deploy SLAM-equipped drones as digital cartographers, mapping deforestation in 3D with centimeter detail—a technique rooted in Mur-Artal’s early work [1]. Meanwhile, Kaufmann’s racing drones aren’t just for show; they stress-test AI’s reflexes, hinting at futures where drones outmaneuver humans in high-stakes scenarios [9]. Looking ahead, researchers speculate about drones inspired by nature—think dragonfly-like agility—or quantum computing breakthroughs that crunch navigation data faster. Yet challenges linger: creating universal safety standards, shielding AI from hackers, and harnessing solar or kinetic energy mid-flight.

The drone revolution is a mosaic of disciplines. Vision algorithms grant them sight, adaptive controllers mimic balance, and energy hacks stretch their stamina. From blockchain-armored swarms to disaster-ready mappers, each innovation inches drones closer to ubiquity. But the path forward demands collaboration: engineers refining hardware, coders fortifying AI, and policymakers drafting airspace rules. Only then can drones evolve from niche tools to trusted partners in our skies.

CHAPTER 2

INTRODUCTION

The development of UAV technology has surged in recent years, leading to advancements that enable unmanned aerial vehicles (UAVs) to perform a wide array of tasks with precision and efficiency. This project focuses on creating an advanced flight controller and flight computer specifically designed for quadcopters, leveraging their unique capabilities to navigate complex environments.

A quadcopter is a type of UAV characterized by its four rotors, which provide thrust and maneuverability. The design allows for vertical take-off and landing, as well as hovering capabilities, making it ideal for various applications such as aerial photography, surveillance, and disaster response. The mechanics of a quadcopter involve intricate dynamics governed by a six-degree-of-freedom model, which encompasses translational movements (x, y, z axes) and rotational movements (roll, pitch, yaw).

To implement the flight controller, we start by formulating a mathematical model that accurately describes the quadcopter's motion. This includes deriving equations of motion based on the Newton-Euler approach, which accounts for the forces exerted by the rotors and the resultant torques that affect the vehicle's orientation. The mathematical model will include key parameters such as rotor thrust, aerodynamic drag, and inertia, allowing us to simulate the quadcopter's behavior under various conditions.

The flight controller will be developed using a combination of control strategies, including PID control for basic stabilization and . PID controllers will help manage the quadcopter's stability by adjusting the speed of the rotors based on feedback from sensors, such as inertial measurement units (IMUs) and GPS.

2.1 CLASSIFICATION

UAVs are classified based on parameters such as size, range, weight, engine type, and configuration. The primary UAV categories include fixed-wing, fixed-wing hybrid, single-rotor, and multirotor UAVs. Each of these configurations has unique advantages and limitations that influence their suitability for specific missions.

MULTICOPTERS

Multicopters, also known as multirotor drones, use multiple propellers to generate lift and thrust. They offer superior maneuverability and are widely used in applications that require precise control, such as aerial photography, inspections, and surveillance. Common configurations include tricopters, quadcopters, hexacopters, and octocopters, with quadcopters being the most popular due to their simplicity, cost-effectiveness, and agility. However, multicopters are limited by high energy consumption and relatively short flight durations.

Multicopters are particularly advantageous in environments requiring high agility and stable hovering capabilities. They are commonly used for infrastructure inspections, such as power lines and bridges, where precise positioning is necessary. Additionally, their compact size makes them suitable for urban operations, including law enforcement surveillance and event monitoring. However, their relatively short endurance limits their application for long-range missions, making them less effective for tasks requiring extended flight times.

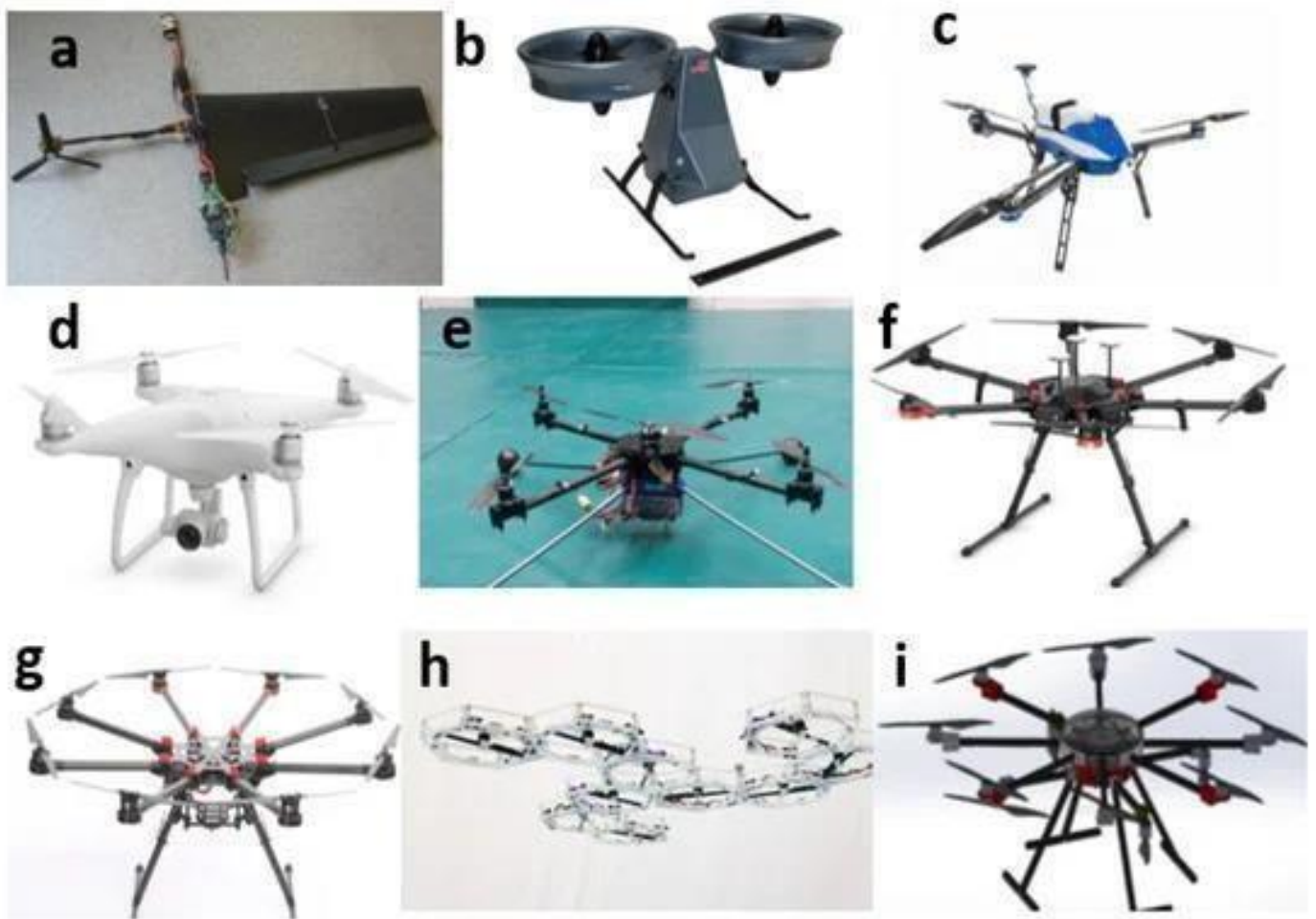


Fig 1: Multirotor UAVs: (a) Monocopter (b) Twincopter, (c) Tricopter , (d) Quadcopter(e) Pentacopter (f) Hexacopter (g) Octocopter, (h) Decacopter, (i) Dodecacopter

2.2 Fixed-Wing UAVs

Fixed-wing UAVs generate lift through aerodynamic surfaces, while thrust is provided by one or more propellers. These UAVs are known for their high endurance, efficiency, and range, making them suitable for applications such as mapping, reconnaissance, and cargo transportation. The most common configuration is the tail-aft design, featuring wings, a fuselage, and an empennage at the rear. Fixed-wing UAVs generally require runways or catapult launchers for takeoff and landing, limiting their operational flexibility. Fixed-wing UAVs are widely used for surveillance and remote sensing applications due to their extended flight durations and ability to cover large areas. Their aerodynamic efficiency allows them to stay airborne for extended periods, making them ideal for border patrol, search and rescue, and environmental monitoring. However, their inability to hover or perform vertical takeoff and landing restricts their usability in confined spaces, necessitating additional launch and recovery infrastructure.

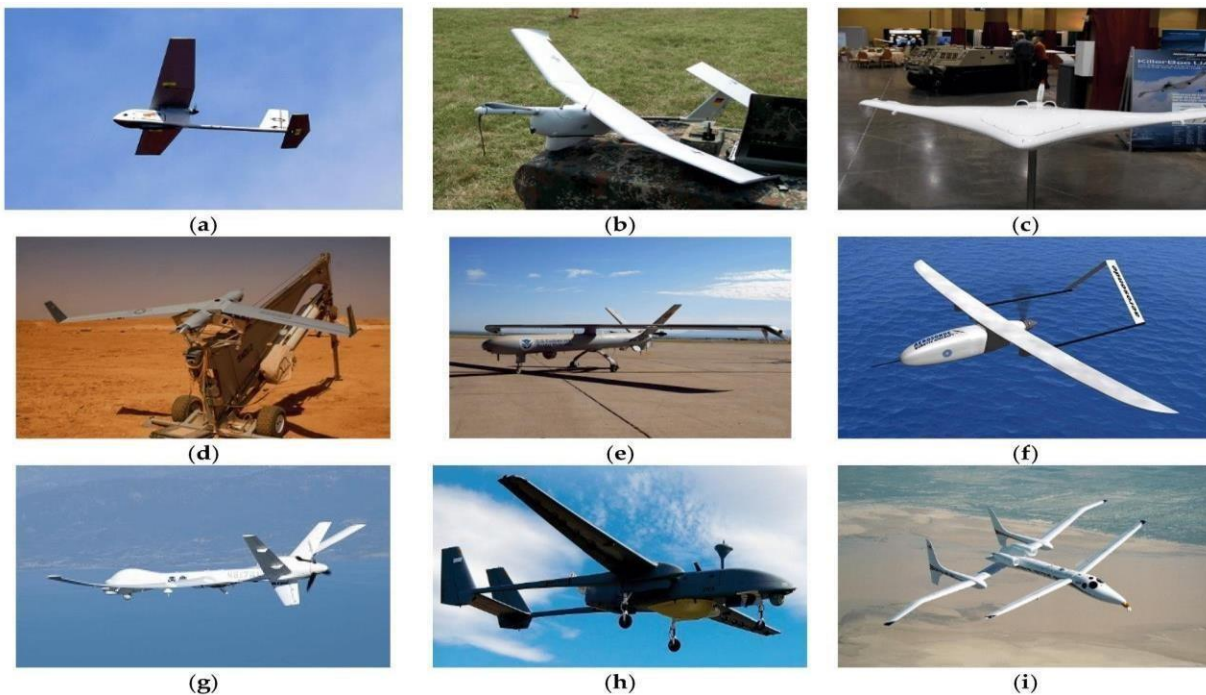


Fig 2 : Examples of fixed wing uavs that have been considered in the different aerodynamic configuration categories (a) AeroVironment RQ-11 Raven, conventional tail RPAS; (b) EMT Aladin, T-tail RPAS; (c) Northrop Grumman KillerBee, tailless RPAS; (d) Boeing Insitu ScanEagle, also considered in the tailless category; (e) Elbit Hermes 450, V-tail RPAS; (f) AAI Aerosonde, V-tail RPAS; (g) General Atomics Altair, Y-tail RPAS; (h) IAI Heron, H-tail RPAS; (i) Scaled Composites Proteus, included in the tandem wing category.

2.3 Fixed-Wing VTOL UAVs

Fixed-wing vertical takeoff and landing (FW-VTOL) UAVs combine the benefits of fixed-wing endurance with the maneuverability of multicopters. The most common design is the separate lift-thrust configuration, also known as the quad-plane, where four lifting propellers enable vertical takeoff and landing, while a separate propulsion system provides forward flight capability. FW-VTOL UAVs eliminate the need for runways and are ideal for operations requiring both long-range travel and hovering capabilities. However, optimizing the VTOL propulsion system is critical to avoid excessive weight penalties that could reduce performance.

FW-VTOL UAVs are becoming increasingly popular in commercial and defense applications due to their ability to operate in diverse environments. They are particularly beneficial for medical supply deliveries in remote areas, where the ability to take off and land vertically is a key advantage. Additionally, they offer significant benefits for military reconnaissance missions, enabling rapid deployment without the need for an airstrip. Emerging technologies such as tilt-wing and tilt-rotor configurations aim to improve the efficiency of FW-VTOL UAVs by enhancing their transition between vertical and forward flight modes.

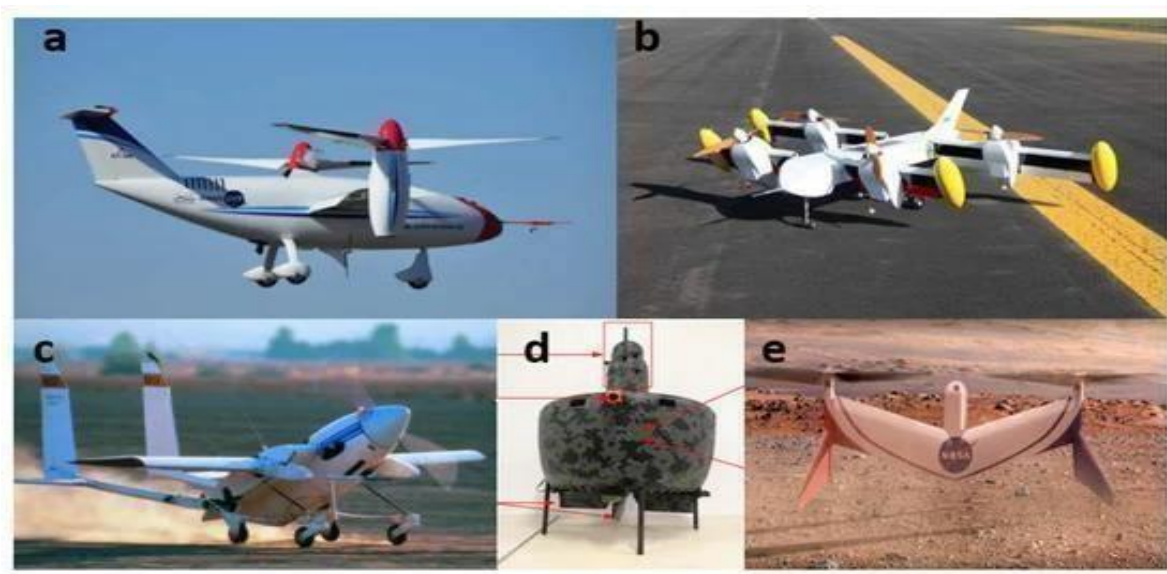


Fig 3 : Hybrid UAVs: (a) tilt-rotor, (b) tilt-wing, (c) tilt-body, (d) ducted fan, (e) tail-sitter

2.4 OBJECTIVE

The primary objective of this project is to create an innovative flight control system and flight computer for Unmanned Aerial Vehicles (UAVs) that can autonomously navigate, avoid obstacles, track humans, and execute complex missions. This endeavor focuses on two core areas:

1. **Flight Controller Development:** We aim to design a cutting-edge flight controller that ensures stable and responsive flight behavior. This involves utilizing advanced control strategies such as PID, adaptive control, and robust feedback mechanisms to effectively manage the quadrotor's dynamics. The challenge lies in addressing the non-linear characteristics of UAVs while maintaining energy efficiency and operational reliability.
2. **Flight Computer Implementation:** The project will also involve developing a sophisticated flight computer capable of processing data from various onboard sensors, including LiDAR, cameras, and GPS. This system will facilitate real-time perception of the environment, allowing the UAV to make informed decisions about navigation, obstacle avoidance, and mission execution.
3. Once the theoretical framework is established, the next phase involves transitioning from simulation to implementation. We will utilize platforms such as **Pixhawk** or other microcontroller-based systems to integrate the flight controller with the quadcopter's hardware. This includes programming the flight control algorithms and ensuring seamless communication with the onboard sensors.
4. The flight computer will further enhance the quadcopter's capabilities by processing data from multiple sensors. This includes integrating information from the IMU for orientation, GPS for positioning, and additional sensors like ultrasonic or LiDAR for obstacle detection. By employing sensor fusion techniques, we can create a comprehensive situational awareness model that allows the UAV to navigate autonomously in real time.
5. In addition to basic flight control, the project aims to implement sophisticated features such as obstacle avoidance, human tracking, and autonomous mission planning. The flight computer will use algorithms that analyze sensor data to identify and avoid obstacles, track human movements for applications like search and rescue, and execute pre-defined missions with minimal human intervention.

By the end of this project, we aim to develop a fully functional quadcopter equipped with an advanced flight controller and intelligent flight computer. This system will not only demonstrate stable flight and maneuverability but also showcase the ability to operate autonomously in dynamic environments, setting the stage for future innovations in UAV applications. The insights gained from this project could contribute significantly to the growing field of unmanned aerial systems, offering new solutions for challenges across various industries.

2.5 MOTIVATION

The motivation behind this project stems from the growing significance of UAVs in a wide range of applications, including emergency response, environmental monitoring, and surveillance. With the potential to revolutionize how we approach complex tasks, UAVs can enhance operational efficiency and safety in scenarios where human presence may be risky.

Moreover, as UAV technology continues to advance, there is a pressing need for improved flight control systems that can handle the intricacies of autonomous operations, especially in dynamic environments. By tackling challenges such as energy efficiency, navigation in constrained spaces, and real-time data processing, this project aims to contribute to the development of smarter, more capable UAVs.

In essence, this project not only seeks to advance the technical capabilities of UAVs but also aspires to empower future innovations that leverage aerial robotics for societal benefits. The journey of creating a sophisticated flight control system will pave the way for groundbreaking applications that can transform industries and improve lives.

CHAPTER 3

CONFIGURATION OF UAVs

IMPLEMENTATION OF QUADROTOR CONFIGURATION

Control mixing means assigning the right stabilization for each of axis to the motors or servos outputs, so that all of them work together to keep the vehicle stable during flight. This is handled in the `controlMixer()` function, where the desired throttle (`thro_des`), and the roll, pitch, and yaw PID outputs are combined and assigned to the corresponding scaled motor or actuator commands. These command values should stay within a range of 0 to 1, because `thro_des` is already scaled between 0 and 1, and with properly tuned PID controllers, the roll, pitch, and yaw PID values will generally stay between -1 and 1. A standard X-configuration quadcopter, an example of this mixing is already provided in the default code. In that setup, motor 1 is the front left, motor 2 is the front right, motor 3 is the back right, and motor 4 is the back left.

The best way to figure out the correct assignments and the direction (positive or negative) of the roll, pitch, and yaw PID values is through trial and error—**with the propellers removed** for safety. example, if there is a motor located along the centerline at the front (nose) of the aircraft, you can assume that speeding up or slowing down that motor will mainly affect pitch. So, only the pitch PID (and throttle, if needed) should be assigned to that motor. In the case of a conventional fixed-wing aircraft with two aileron servos, one servo should get a `+roll_PID` signal and the other should get `-roll_PID`. Keep in mind that the sign might need to be flipped depending on how the servos are mounted and oriented.

3.1 CODE IMPLEMENTATION FOR QUADROTOR CONFIGURATION

M1 command = thro des - pitch PID+ yaw PID + roll PID ;

M2 command = thro des - yaw PID - pitch PID - roll PID ;

M3 command = thro des + pitch PID - yaw PID + roll PID ;

M4 command = thro des + pitch PID + roll PID - yaw PID;

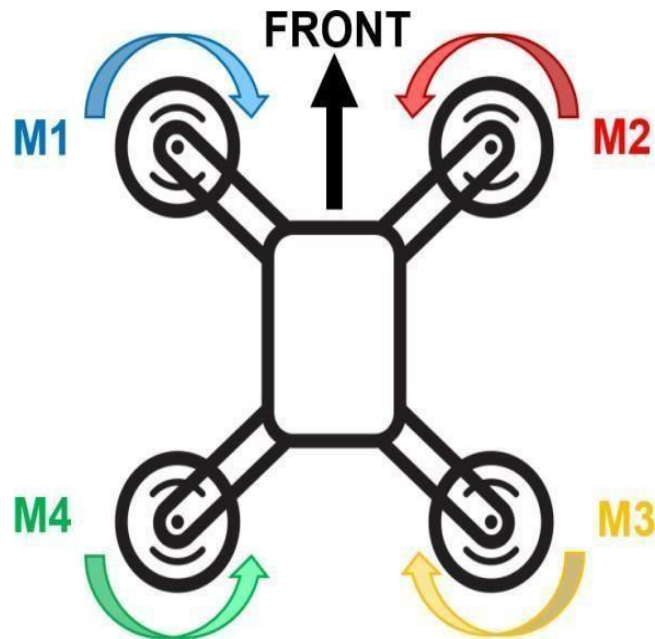


Fig 4 : Motor configuration for quadrotor

3.2 VTOL CONFIGURATION

This section only covers the basics, but there are many possibilities when it comes to advanced VTOL configurations—especially when the vehicle’s behavior changes depending on the flight mode. In the code, channel 6 isn’t used by default, which means we can take advantage of it inside the `controlMixer()` function to switch between different mixing setups. For example, you could use two different mixing configurations (or even three if you’re using a 3-position switch, or technically almost unlimited combinations if you assign a slider to channel 6 on your radio). In the examples below, we’ll assume that channel 6 is connected to a two-position switch, where the PWM signal toggles between a low value of 1000 and a high value of 2000. With a simple `if` statement in the code, you can easily switch between two dynamic flight modes—like using a quadcopter setup for hovering, and switching to a fixed-wing airplane configuration (with one motor, two ailerons, and an elevator, all with no stabilization) for forward flight.

[23] CODE IMPLEMENTATION FOR VTOL CONFIGURATION

```
float pitchAmount = 0.7;
float pitchOffset = 0.5;
float rollAmount = 0.65;
float rollOffsetLeft = 0.5;
float rollOffsetRight = 0.58;
if (channel 6 pwm > 1500) { //hover mode
M1 command scaled = thro des - pitch PID + roll PID + yaw PID; //front left
M2 command scaled = thro des - pitch PID - roll PID - yaw PID; //front right
M3 command scaled = thro des + pitch PID - roll PID + yaw PID; //back right
M4 command scaled = thro des + pitch PID + roll PID - yaw PID; //back left
M5 command scaled = 0; //turn off forward flight motor in hover
S1 command scaled = pitchOffset; //elevator servo, centered and not moving in
hover
S2 command scaled = rollOffsetLeft; //left aileron, centered and not moving in
hover
S3 command scaled = rollOffsetRight;
//right aileron, centered and not moving in hover }
else if (channel 6 pwm < 1500)
M1 command scaled = 0; //turn off in forward flight
M2 command scaled = 0; //turn off in forward flight
M3 command scaled = 0; //turn off in forward flight
M4 command scaled = 0; //turn off in forward flight
M5 command scaled = thro des; //direct throttle control
S1 command scaled = pitchAmount*pitch passthru + pitchOffset; //elevator
S2 command scaled = rollAmount*roll passthru + rollOffsetLeft; //left aileron
S3 command scaled = rollAmount*roll passthru + rollOffsetRight; //right aileron }
```

3.3 METHODOLOGY

Unmanned aerial vehicles (UAVs) have become increasingly important in various civilian and military applications due to their ability to operate without a human pilot onboard. UAVs can perform critical operations without risking human safety and often operate more cost-effectively than manned systems. These aircraft can be remotely piloted using a ground control station or autonomously controlled via onboard sensors, microprocessors, and other electronic components. UAVs are used in applications ranging from remote sensing, surveillance, and transport to search and rescue missions. The increasing development of UAV technology has led to the emergence of various UAV configurations, each optimized for specific applications and performance requirements.

UAVs operate through a complex system architecture that includes navigation, communication, and control systems. A typical UAV system consists of multiple components such as ground control stations (GCS), satellites, onboard processors, and wireless communication modules. These elements work together to provide real-time control, data transmission, and autonomous capabilities, allowing UAVs to function in various environments with minimal human intervention. The development of UAV technology has been significantly influenced by advancements in artificial intelligence (AI), sensor technology, and power systems, making UAVs more capable and versatile than ever before.

Despite the increasing level of automation in UAVs, most systems still rely on human-assisted remote control. Piloting a UAV manually is challenging, requiring intensive training and experience to ensure safe and efficient operation. Manual control is prone to inefficiencies, human errors, and operational constraints, making autonomous capabilities highly desirable. Current UAVs integrate autopilot features; however, they still face significant limitations, including restricted battery life, limited autonomy, and challenges with landing accuracy. Computer vision techniques have been explored to enhance landing precision, but these require additional computational resources and protocols that are not widely available for commercial drones. Furthermore, UAVs face regulatory and operational challenges, as different countries impose strict guidelines on UAV deployment, particularly in urban areas.

The increasing demand for UAVs in commercial and industrial applications has led to the need for optimized UAV designs. UAVs vary widely in terms of specifications, equipment, size, range, and shape. The market offers different configurations, including fixed-wing UAVs, multicopters, fixed-wing VTOL (FW-VTOL) drones, and single-rotor UAVs, each catering to distinct operational requirements. UAVs are also equipped with various sensors, GPS modules, stabilizers, and cameras, enabling them to perform specialized tasks which are becoming an essential tool in many industries, including agriculture, construction, environmental monitoring, and emergency response. With different designs tailored for various applications, UAVs have revolutionized aerial data collection, enabling efficient and accurate operations. For example, UAVs are widely used in agriculture for crop monitoring, pesticide spraying, and soil analysis. In disaster management, UAVs assist in assessing damage, locating survivors, and delivering emergency supplies. Similarly, in urban planning, UAVs provide high-resolution aerial imagery that helps in infrastructure development.

With the continued advancements in UAV technology, further research and innovation are necessary to enhance UAV performance, efficiency, and adaptability in different environments.

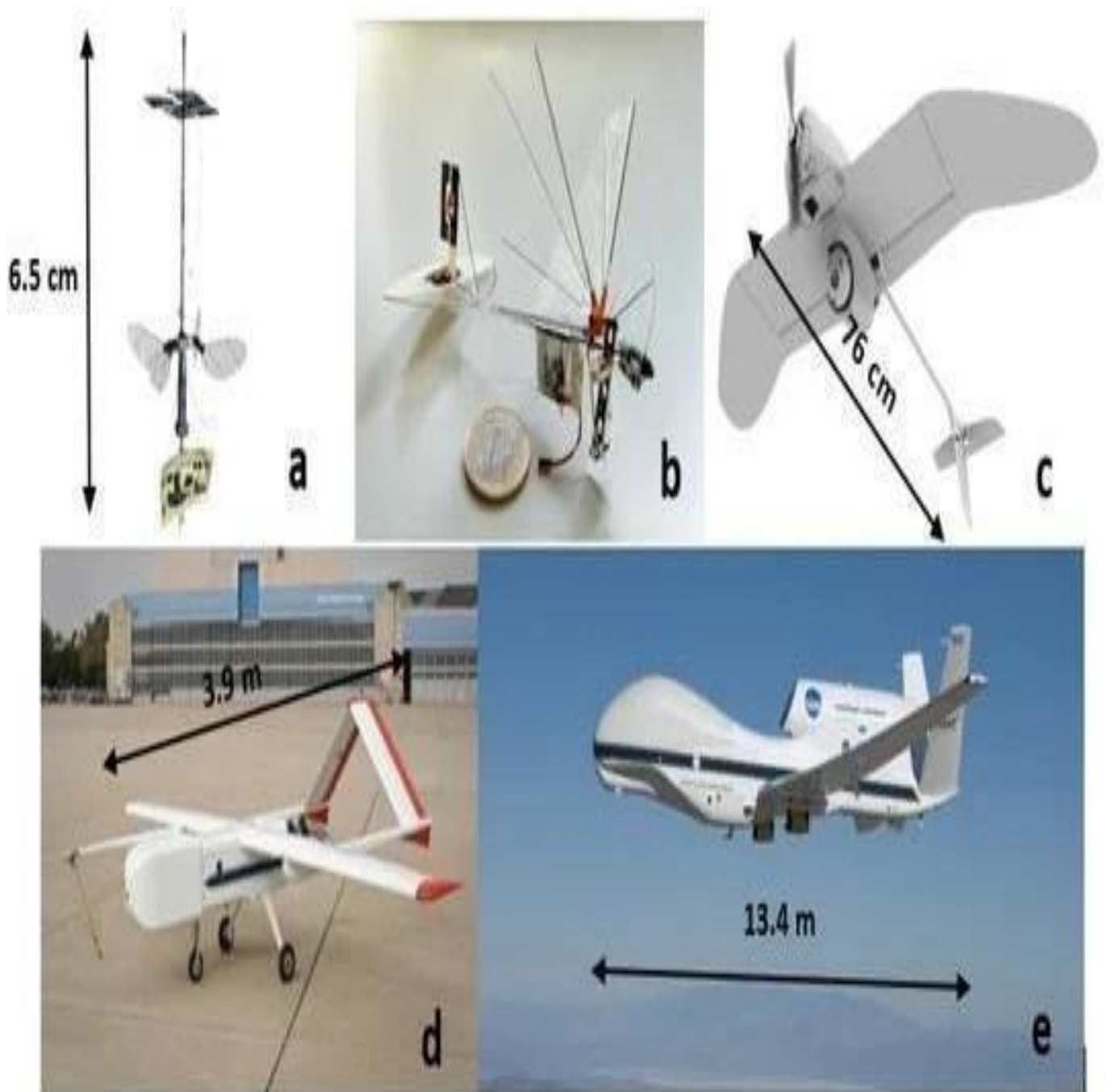


Fig 5 : Different UAV classes based on size: (a) ultra-small; (b) very small (c) small UAV; (d) medium UAV; (e) large UAV.

CHAPTER 4

MODEL OF QUADROTOR, ALTITUDE AND ORIENTATION CONTROL

4.1 Quadrotor absolute angular velocity in Body fixed frame:

Reference frame R ground

$$R \rightarrow R_1 \text{ Yaw } k = k_1$$

$$R_1 \rightarrow R_2 \text{ Pitch } j_1 = j_2$$

$$R_2 \rightarrow R_3 \text{ Roll } i_2 = i_3$$

$${}^R_{R_3} \omega = \dot{\phi} i_3 + \dot{\theta} j_2 + \dot{\psi} i_1$$

$${}^R_{R_3} \omega = {}^{R_2}_{R_3} \omega + {}^{R_1}_{R_2} \omega + {}^R_{R_1} \omega$$

The diagram illustrates the sequence of reference frames used to describe the **absolute angular velocity of a quadrotor** in its **body-fixed frame**. It starts from the **inertial reference frame** labeled as **R**, which represents the ground or world frame with fixed axes (x, y, z). From this frame, the quadrotor undergoes three sequential rotations: **yaw (ψ)**, **pitch (θ)**, and **roll (ϕ)**. Each rotation results in a new intermediate reference frame.

The first rotation is **yaw (ψ)** about the **z-axis**, transitioning from the inertial frame **R** to frame **R₁**. This rotation aligns the quadrotor's heading with its intended direction. The second rotation is **pitch (θ)** about the **new y-axis** (from **R₁**), leading to frame **R₂**. This rotation causes the quadrotor to tilt forward or backward. The final rotation is **roll (ϕ)** about the **x-axis** of frame **R₂**, resulting in frame **R₃**, which is the final **body-fixed frame** of the quadrotor.

In this context, **R₁**, **R₂**, and **R₃** represent intermediate reference frames after each respective rotation: yaw, pitch, and roll. The angular velocity of the quadrotor in the body-fixed frame is expressed as a combination of roll rate ($\dot{\phi}$), pitch rate ($\dot{\theta}$), and yaw rate ($\dot{\psi}$), each aligned with the corresponding axes of the rotated frames. Mathematically, the total angular velocity is composed by summing the contributions of each rotation, transformed into the final body frame. This structured approach allows precise modeling of the

quadrotor's orientation and dynamics during flight.

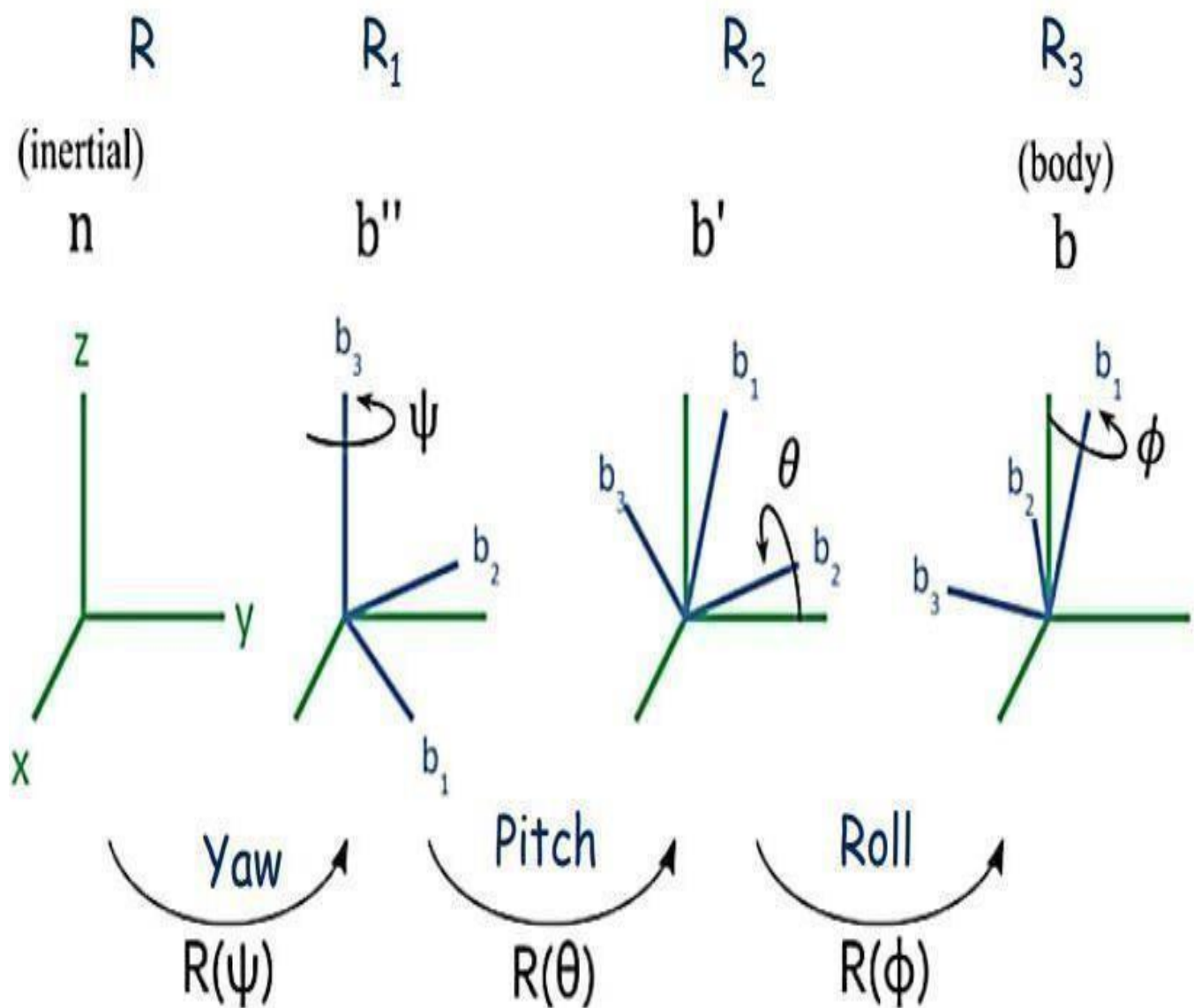


Fig 6 : Reference frame with reference to Raw Pitch Roll angles.

Addition Theorem for Angular Velocity

All Simple Angular motion

$${}^R_3\omega = i3(\dot{\phi} - \psi''' \sin\theta'') + j3(\theta''' \cos\phi + \psi''' \cos\theta'' \sin\phi) + k3(-\theta''' \sin\phi + \psi''' \cos\theta'' \cos\phi)$$

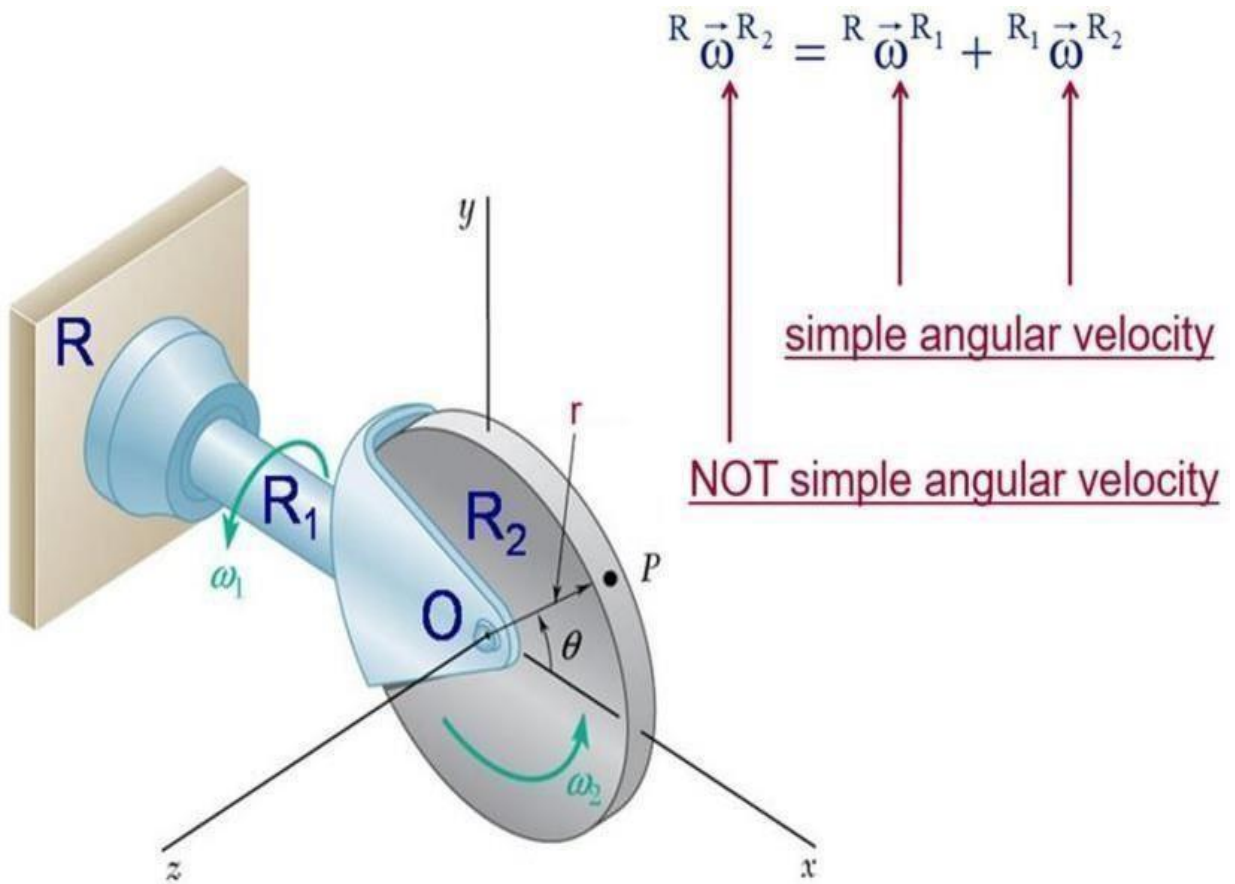


Fig 7 : Angular velocity of quadcopter on ground with reference to Raw Pitch Roll reference frame.

4.2 Roll-Pitch-Yaw

When we rotate the axis X by angle ϕ , axis Y by angle θ and axis Z by angle ψ then axis b1, b2 and b3 is related to x, y and z as follows:

$$\begin{aligned}
 b_1 &= 1 \cdot x + 0 \cdot y + 0 \cdot z \\
 b_2 &= 0 \cdot x + \cos \Phi \cdot y + \sin \Phi \cdot z \\
 b_3 &= 0 \cdot x - \sin \Phi \cdot y + \cos \Phi \cdot z
 \end{aligned}$$

$$\begin{aligned}
 b_1 &= \cos \theta \cos \psi \cdot x + \cos \theta \sin \psi \cdot y - \sin \theta \cdot z \\
 b_2 &= -\sin \Phi \cos \psi + \sin \Phi \sin \theta \cos \psi \cdot x + \cos \Phi \cos \psi + \sin \Phi \sin \theta \sin \psi \cdot y + \sin \Phi \cos \theta \cdot z \\
 b_3 &= \sin \Phi \sin \psi + \cos \Phi \sin \theta \sin \psi \cdot x - \sin \Phi \cos \psi + \cos \psi \sin \theta \cos \Phi \cdot y + \cos \psi \cos \theta \cdot z
 \end{aligned}$$

Here Φ is roll angle, θ is pitch angle and ψ is yaw angle

4.3 Body-Fixed Axes

In a quadrotor (a type of drone or UAV), the **body-fixed axes** refer to a coordinate system that is fixed to the frame of the quadrotor itself. This coordinate system moves and rotates with the quadrotor, providing a reference for describing its orientation, angular velocity, and forces acting on it.

Components of Body-Fixed Axes

The body-fixed axes typically consist of three mutually perpendicular axes, labeled as:

1. **X-axis:** Often points forward in the direction the quadrotor's nose (front) is facing.
2. **Y-axis:** Points to the right side of the quadrotor when viewed from above.
3. **Z-axis:** Points downward, perpendicular to the quadrotor's plane of rotation.

$$\begin{bmatrix} w_{roll} \\ w_{pitch} \\ w_{yaw} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\Phi & \sin\Phi\cos\theta \\ 0 & -\sin\Phi & \cos\Phi\cos\theta \end{bmatrix} \begin{bmatrix} \dot{\psi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

INITIAL REFERENCE FRAME

The **Initial Reference Frame** of a quadcopter is the fixed coordinate system relative to which all the motion of the quadcopter is measured. This frame is typically defined using **three orthogonal axes** and serves as a global reference for describing the quadcopter's position, orientation, and movement.

$$\begin{bmatrix} \dot{\psi} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 1 & \tan\theta\sin\Phi & \tan\theta\cos\Phi \\ 0 & \cos\Phi & -\sin\Phi \end{bmatrix} \begin{bmatrix} w_{roll} \\ w_{pitch} \\ w_{yaw} \end{bmatrix}$$

Here Φ is roll angle, θ is pitch angle and $\dot{\psi}$ is yaw angle.

4.4 NEWTON EULER EQUATIONS

The **Initial Reference Frame** of a quadcopter is the fixed coordinate system relative to which all the motion of the quadcopter is measured. This frame is typically defined using **three orthogonal axes** and serves as a global reference for describing the quadcopter's position, orientation, and movement.

$$\Sigma F = \frac{d}{dt} L = M \frac{d}{dt} v$$

$$\Sigma M = \frac{d}{dt} H$$

Here

F is the force on the quadcopter

L is the angular momentum of the quadcopter

H is the absolute angular momentum

Angular Momentum

$$\begin{aligned} {}^{R3}H_{x3} &= I_{x3} \dot{\phi} - \psi'' \sin \theta'' \\ {}^{R3}H_{y3} &= I_{y3} \dot{\theta} \cos \phi + \psi'' \cos \theta'' \sin \phi \\ {}^{R3}H_{z3} &= I_{z3} \dot{\psi} - \theta'' \sin \phi + \psi'' \cos \theta'' \cos \phi \end{aligned}$$

H is the absolute angular momentum

I is the moment of inertia with respect to Body Fixed axes

w is the absolute angular velocity

Differentiation of a vector in two Reference Frames

If R and R_1 are any 2 reference frames, the first-time derivatives of any vector β (not fixed in either R or R_1) in R and R_1 are related to each other as follows:

$$\frac{d\beta}{dt}^R = \frac{d\beta}{dt}^{R_1} + (\omega^{R_{R_1}} \times \beta)$$

Equations of Motion

$$\Sigma F = \frac{d}{dt} L = M \frac{dv}{dt}^R$$

$$\frac{dv^c}{dt}^R = \frac{dv^c}{dt}^{R_3} + (\omega^{R_{R_3}} \times v^c)$$

$$v^c = u i_3 + v j_3 + w k_3$$

$$\frac{dv^c}{dt}^R = i \frac{d}{dt} \left[u \dot{\phi} + w(\ddot{\theta} \cos \phi + \dot{\psi} \cos \theta \sin \phi) + j [v \dot{\theta} - w(\dot{\phi} - \dot{\psi} \sin \theta)] + u(-\ddot{\theta} \sin \phi + \dot{\psi} \ddot{\theta} \cos \theta \cos \phi) \right] + k_3 [w \dot{\theta} + v(\dot{\phi} - \dot{\psi} \sin \theta) - u(\ddot{\theta} \cos \phi + \dot{\psi} \ddot{\theta} \cos \theta \cos \phi)]$$

$$\Sigma M = T_\phi i_3 + T_\theta j_3 + T_\psi k_3$$

$$= l_k (w^2 - w^2) i_3 - l_k (w^2 - w^2) j_3 + b (w^2 + w^2 - w^2 - w^2) k_3$$

$$\frac{dH}{dt}^{R_3} = i I_{x_3} (\dot{\phi} - \dot{\psi} \sin \theta - \dot{\psi} \dot{\theta} \cos \theta)$$

$$+ j_3 I_y (\ddot{\theta} \cos \phi - \ddot{\theta} \dot{\phi} \sin \phi + \dot{\psi} \ddot{\theta} \cos \theta \sin \phi - \dot{\psi} \dot{\phi} \sin \theta \sin \phi + \dot{\psi} \dot{\phi} \cos \theta \sin \phi)$$

$$+ k_3 I_z (-\ddot{\theta} \sin \phi - \ddot{\theta} \dot{\phi} \cos \phi + \dot{\psi} \ddot{\theta} \cos \theta \cos \phi - \dot{\psi} \dot{\phi} \sin \theta \cos \phi$$

$$- \dot{\psi} \dot{\phi} \cos \theta \sin \phi)_{R_3}^R w \times H$$

$$= i_3 q r (I_{z_3} - I_{y_3}) + j_3 p r (I_{x_3} - I_{z_3}) + k_3 p q (I_{y_3} - I_{x_3})$$

Here

F is the force on the quadcopter

L is the angular momentum of the quadcopter

H is the absolute angular momentum

SUMMARY

NEWTON EULER EQUATIONS OF MOTION

Here

$$p = \dot{\phi} - \dot{\psi} \sin \theta$$

$$q = \dot{\theta} \cos \phi + \dot{\psi} \cos \theta \sin \phi$$

$$r = -\dot{\theta} \sin \phi + \dot{\psi} \cos \theta \cos \phi$$

- 1) $mg \sin \theta = m[u + wq - vr]$
- 2) $-mg \cos \theta \sin \phi = m[v - wp - ur]$
- 3) $T - mg \cos \theta \cos \phi = m[w + vp - uq]$
- 4) $I_k(\ddot{w}^2 - \dot{w}^2) = I_x \ddot{p} + qr(I_z - I_y)$
- 5) $I_k(\ddot{w}^2 - \dot{w}^2) = I_y \ddot{q} + pr(I_x - I_z)$

4.5 LAGRANGE METHOD

Advantages

- 1) The system of particles and rigid bodies is considered as a whole, rather than being separated into its individual components.
- 2) Problems are formulated in terms of kinetic energy and work, both of which are scalar quantities.
- 3) Forces of constraint that do not perform work are not included. If constraint forces are required, apply the Newton-Euler Equation or use Lagrange Multipliers.
- 4) Use of generalized coordinates, instead of physical coordinates, affords ease and makes the formulation versatile.

Generalized Coordinates

1. Generalized coordinates are basically a set of variables that describe how a system moves, taking into account any restrictions or limitations it has..
2. We say a set of coordinates is independent if, when you fix all of them except one, that last one still has the freedom to vary within a certain range of possibilities. This means the system can still move in some way.
3. The coordinates are considered complete if, by knowing their values, you can fully pinpoint the position of every part of the system for any possible setup the system can have.

Holonomic Constraints

- Consider a system described by n generalized coordinates $q_1, q_2, \dots, q_n$. Suppose that there are m constraint equations of the form:

$$f_i(q_1, \dots, q_n, t) = 0 \quad (i = 1, 2, \dots, m)$$
- Constraints of this type are called holonomic constraints. In principle, these equations allow us to express the m coordinates in terms of the remaining $(n-m)$ coordinates and time. This way, we end up with only as many generalized coordinates as the system's degrees of freedom.

Holonomic Systems:

$$\frac{d}{dt} \left(\frac{\delta T}{\delta \dot{q}_i} \right) - \frac{\delta T}{\delta q_i} = Q_i$$

- Where:

q_i are the generalized coordinates.

Q_i is the generalized force in the direction of the i th coordinate.

T is the kinetic energy of the system.

- For conservative forces (e.g., gravitational, elastic):

$$F = -\Delta V \quad Q_i = -\frac{\delta V}{\delta q_i}$$

$$\frac{d}{dt} \left(\frac{\delta T}{\delta \dot{q}_i} \right) - \frac{\delta T}{\delta q_i} + \frac{\delta V}{\delta q_i} = Q_i \quad i = 1, 2, \dots, n$$

Body-Fixed Axis

Generalized coordinates: $x, y, z, \phi, \theta, \psi$

(Absolute Position and Absolute orientation)

$$\text{Kinetic Energy } T = \frac{1}{2}mv^2 + \frac{1}{2}[I_x \omega_x^2 + I_y \omega_y^2 + I_z \omega_z^2 + I_{xy} \omega_x \omega_y + I_{yz} \omega_y \omega_z + I_{zx} \omega_z \omega_x]$$

$$= \frac{1}{2}m[\dot{x}^2 + \dot{y}^2 + \dot{z}^2] + \frac{1}{2}[I_{x3}p^2 + I_{y3}q^2 + I_{z3}r^2]$$

Here- $p = \dot{\phi} - \dot{\psi} \sin \theta$

$$q = \dot{\theta} \cos \phi + \dot{\psi} \cos \theta \sin \phi$$

$$r = -\dot{\theta} \sin \phi + \dot{\psi} \cos \theta \cos \phi$$

$$\text{If } q_i = x \rightarrow \frac{\delta T}{\delta \dot{x}} = m\dot{x} \quad \frac{d}{dt} \left(\frac{\delta T}{\delta \dot{x}} \right) = m\ddot{x} \quad \frac{\delta T}{\delta x} = 0 \quad \frac{\delta V}{\delta x} = 0$$

$$\text{If } q_i = y \rightarrow \frac{\delta T}{\delta \dot{y}} = m\dot{y} \quad \frac{d}{dt} \left(\frac{\delta T}{\delta \dot{y}} \right) = m\ddot{y} \quad \frac{\delta T}{\delta y} = 0 \quad \frac{\delta V}{\delta y} = 0$$

$$\text{If } q_i = z \rightarrow \frac{\delta T}{\delta \dot{z}} = m\dot{z} \quad \frac{d}{dt} \left(\frac{\delta T}{\delta \dot{z}} \right) = m\ddot{z} \quad \frac{\delta T}{\delta z} = 0 \quad \frac{\delta V}{\delta z} = 0$$

$$Q_{x,nc} = 0 \quad Q_{y,nc} = 0 \quad Q_{z,nc} = 0$$

On Solving the above equations:

$$1) m\ddot{x} = T(\sin \phi \sin \psi + \cos \phi \sin \theta \cos \psi)$$

$$2) m\ddot{y} = T(-\sin \phi \cos \psi + \cos \phi \sin \theta \sin \psi)$$

$$3) m\ddot{z} = T(\cos \phi \cos \theta) - mg$$

$$\text{If } q_i = \psi'' \rightarrow \frac{\delta T}{\delta \psi'''} = I_x p \left(\frac{\delta p}{\delta \psi''} \right) + I_{y3} q \left(\frac{\delta q}{\delta \phi} \right) + I_{z3} r \left(\frac{\delta r}{\delta \theta''} \right) \frac{\delta V}{\delta \theta''} = 0 \quad Q_{\psi'',nc} = b(w_2^2 + w_4^2 - w_3^2 - w_1^2)$$

Let-

$$\mathbf{n} = \begin{bmatrix} \Phi \\ \theta \\ \psi \end{bmatrix}$$

$$J \frac{d^2 \mathbf{n}}{dt^2} + C \frac{d \mathbf{n}}{dt} = \mathbf{B} \quad \mathbf{B} = \tau \begin{bmatrix} \tau \Phi \\ \theta \\ \tau \psi \end{bmatrix}$$

Here $s_{\theta''}$ is $\sin \theta''$, s_ϕ is $\sin \phi$, $s_{\psi''}$ is $\sin \psi''$, $c_{\theta''}$ is $\cos \theta''$, c_ϕ is $\cos \phi$, $c_{\psi''}$ is $\cos \psi''$

$$\mathbf{J} = \begin{bmatrix} I_{xx} & \mathbf{0} & -I_{xx} S_\theta \\ \mathbf{0} & I_{yy} C_\Phi + I_{zz} S_\Phi & (I_{yy} - I_{zz}) C_\Phi S_\Phi C_\theta \\ -I_{xx} S_\theta & (I_{yy} - I_{zz}) C_\Phi S_\Phi C_\theta & I_{xx} S_\theta + I_{yy} S_\Phi C_\theta + I_{zz} C_\Phi C_\theta \end{bmatrix}$$

$$-I_{xx} S_\theta \quad (I_{yy} - I_{zz}) C_\Phi S_\Phi C_\theta \quad I_{xx} S_\theta + I_{yy} S_\Phi C_\theta + I_{zz} C_\Phi C_\theta$$

$$C_{11} = 0$$

$$C_{12} = (I_{yy} - I_{zz})(\theta''' C_\phi S_\phi + \psi''' S^2 C_{\theta''}) + (I_{zz} - I_{yy})\psi''' C_\phi^2 C_{\theta''} + I_{xx} \psi''' C_{\theta''}$$

$$C_{13} = (I_{zz} - I_{yy})\psi''' C_\phi S_\phi C_\theta^2$$

$$C_{21} = (I_{zz} - I_{yy})(\theta''' C_\phi S_\phi + \psi''' S_\phi C_{\theta''}) + (I_{yy} - I_{zz})\psi''' C_\phi^2 C_{\theta''} + I_{xx} \psi''' C_{\theta''} C_{22} = (I_{zz} - I_{yy})\phi' C_\phi S_\phi$$

$$C_{23} = -I_{xx} \psi''' S_{\theta''} C_{\theta''} + I_{yy} \psi''' S^2 S_{\theta''} C_{\theta''} + I_{zz} \psi' C_\phi^2 S_\theta C_{\theta''}$$

$$C_{31} = (I_{yy} - I_{zz})\psi''' C_\phi^2 S_\phi C_\phi - I_{xx} \theta''' C_{\theta''}$$

$$C_{32} = (I_{zz} - I_{yy})(\theta''' C_\phi S_\phi S_{\theta''} + \phi' S^2 C_{\theta''}) + (I_{yy} - I_{zz})\phi' C_\phi^2 C_{\theta''} + I_{xx} \psi''' C_{\theta''} S_{\theta''}$$

$$C_{33} = (I_{yy} - I_{zz})\phi' C_\phi S_\phi C_\theta^2 + I_{xx} \theta''' C_{\theta''} S_{\theta''}$$

4.6 MATLAB SIMULATION

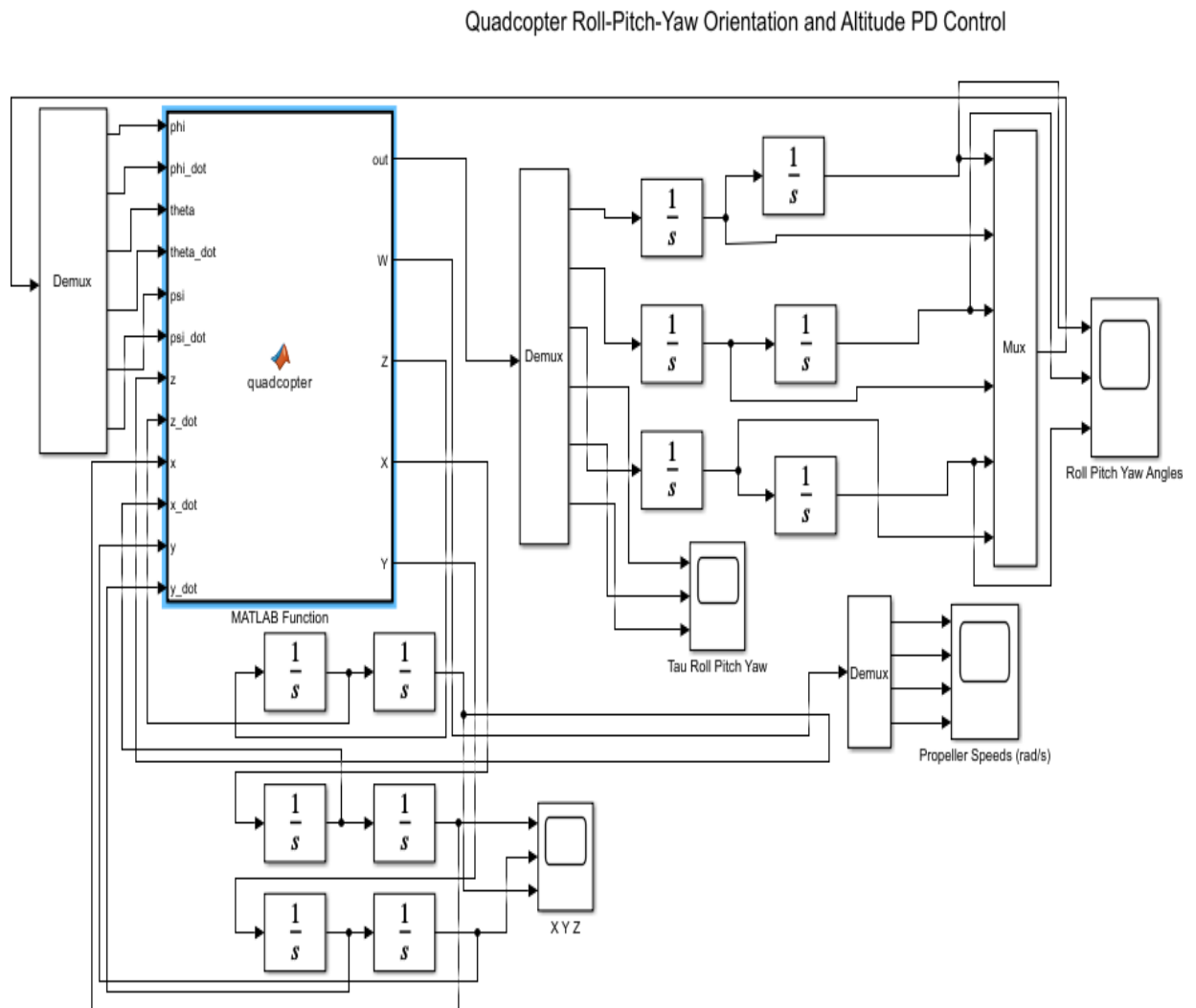


Fig 8 : Quadcopter Roll-Pitch-Yaw Orientation and Altitude PD Control

4.6.1 Purpose of Model

The model could simulate:

- **Altitude Control:** Maintaining or adjusting the vertical position of the quadrotor.
- **Orientation Control:** Managing the angles of roll, pitch, and yaw for stability and navigation.

4.6.2 Components of Model

a. Quadcopter Block

It represents the dynamic model of the quadcopter. Here the inputs given are $\theta, \theta'', \psi'', \psi''', \phi, \phi', X, X', Y, Y', Z, Z'$. This block models the equations of motion of the quadcopter using physics, including its inertial parameters and how forces/torques affect it.

b. Demux Block

It separates a vector signal into individual components. The inputs like position, velocity, and angles are grouped as vectors. The Demux block splits these into separate scalar signals for further processing.

c. Integrator Blocks (1/s)

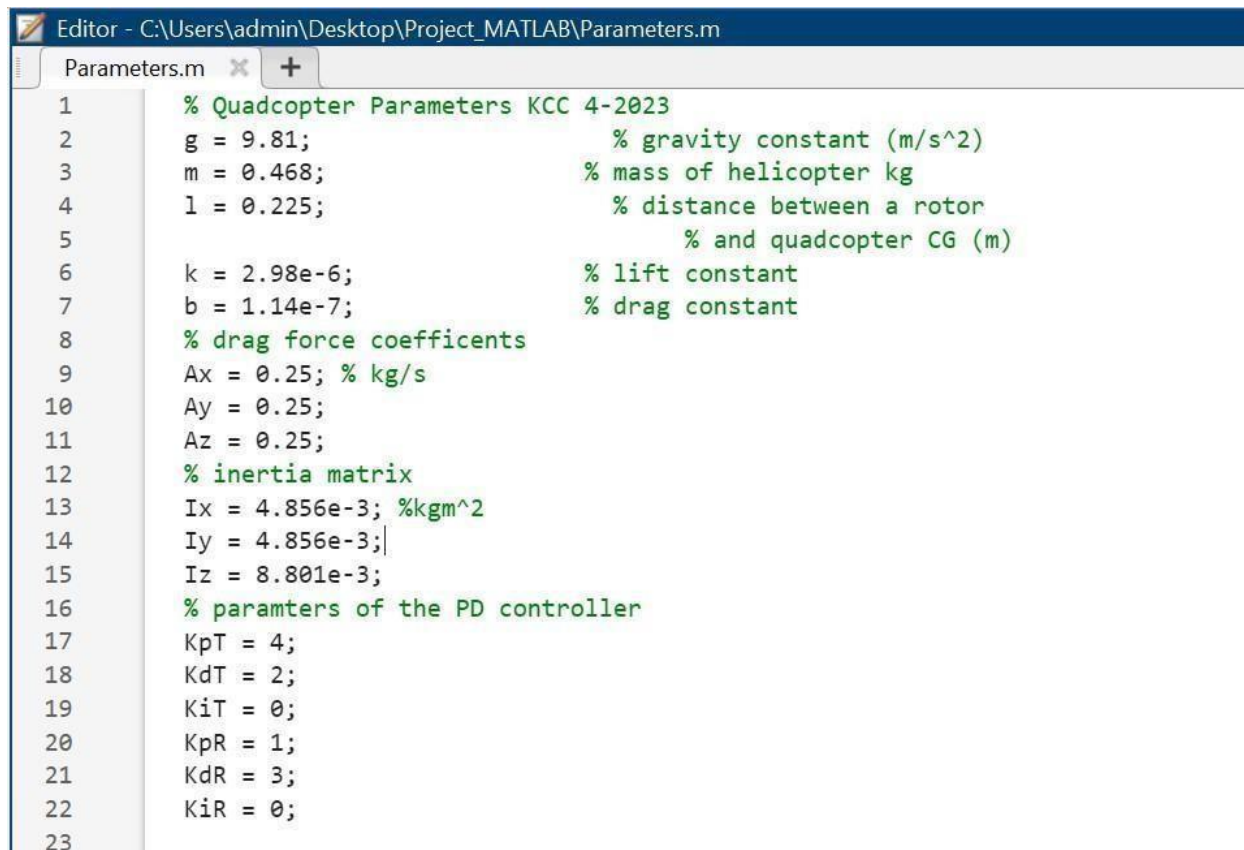
It is used to model integration in continuous-time systems. Each integrator corresponds to a time integral of its input signal. For example, the angular rate (ϕ') is integrated to obtain the roll angle (ϕ), and similarly, for θ'' and ψ'' .

d. PD Controllers

It stabilizes the quadcopter by minimizing the errors in roll, pitch, yaw, and altitude. **Proportional (P):** Reacts to the magnitude of the error and **Derivative (D):** Reacts to the rate of change of the error. The PD controller balances fast response (P-term) with smoothness and stability (D-term).

e. Output Scopes

It Display the results of the simulation for analysis. The **Roll, Pitch, and Yaw Angles** scope displays the attitude (orientation) of the quadcopter over time. The **Tau Roll Pitch Yaw** scope shows the control torques applied to stabilize the quadcopter. The **XYZ** scope visualizes the position or trajectory of the quadcopter in 3D space.



```
Editor - C:\Users\admin\Desktop\Project_MATLAB\Parameters.m
Parameters.m x +
1 % Quadcopter Parameters KCC 4-2023
2 g = 9.81; % gravity constant (m/s^2)
3 m = 0.468; % mass of helicopter kg
4 l = 0.225; % distance between a rotor
5 % and quadcopter CG (m)
6 k = 2.98e-6; % lift constant
7 b = 1.14e-7; % drag constant
8 % drag force coefficients
9 Ax = 0.25; % kg/s
10 Ay = 0.25;
11 Az = 0.25;
12 % inertia matrix
13 Ix = 4.856e-3; %kgm^2
14 Iy = 4.856e-3;
15 Iz = 8.801e-3;
16 % paramters of the PD controller
17 KpT = 4;
18 KdT = 2;
19 KiT = 0;
20 KpR = 1;
21 KdR = 3;
22 KiR = 0;
23
```

Fig 9 : Parameters used in simulation of mathematical model

CHAPTER 5

HARDWARE REQUIREMENTS

The physical flight controller that runs the code is built around a **Teensy 4.0 microcontroller**, paired with an **MPU6050** sensor, which provides 6 degrees of freedom (3-axis gyroscope + 3-axis accelerometer) for inertial measurement. This setup allows the flight controller to read orientation and motion data in real-time, which is essential for stabilization and control of the vehicle.

5.1 Teensy 4.0

The Teensy 4.0 is a powerful and affordable microcontroller, well-suited for robotics, audio applications, and a wide range of Arduino-style projects. It's built around an ARM Cortex-M7 processor running at an impressive 600 MHz, offering plenty of processing power for demanding tasks. The board provides 40 digital input/output pins, all of which support interrupts, and 31 of these pins are capable of generating PWM signals—ideal for controlling motors and servos. On top of that, it includes 3 I2C ports, 3 CAN bus interfaces, and 7 serial ports, giving you a lot of flexibility to connect and communicate with various sensors and devices.

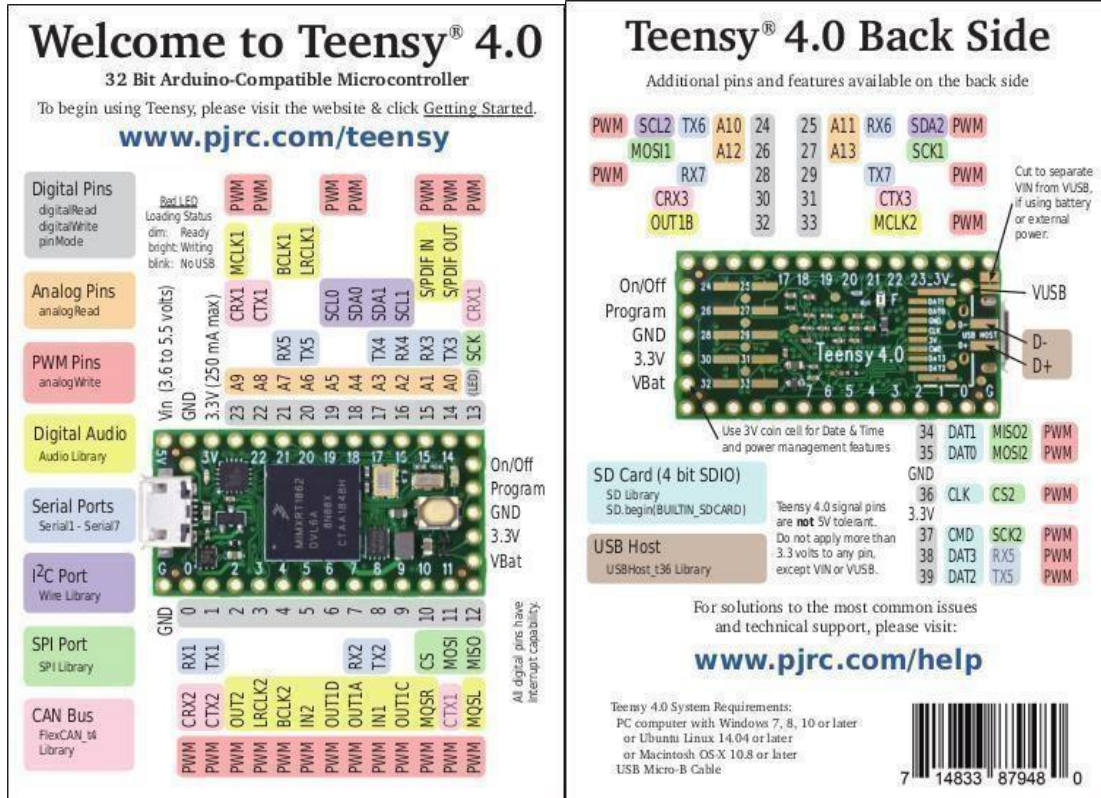


Fig 10 : Teensy 4.0 Development Board

5.2 MPU6050 IMU

The MPU6050 is a six-axis motion tracking device that provides data from a three-axis accelerometer and a three-axis gyroscope. Its biggest advantage is the built-in **Digital Motion Processor (DMP)**, which can offload complex motion processing tasks from the main microcontroller, helping improve performance and efficiency. This sensor board is highly cost-effective, reliable, sufficient measurement precision to effectively stabilize small flying vehicles,



Fig 11: Inertial Measurement Unit

5.3 Hardware Setup of teensy 4.0

The code is designed to support outputs for up to **6 motors and 7 servos**, offering plenty of flexibility for various aircraft configurations. It also accommodates different types of receiver inputs, including **PPM, SBUS**, or a standard **6-channel PWM receiver**, making it compatible with most common radio systems.

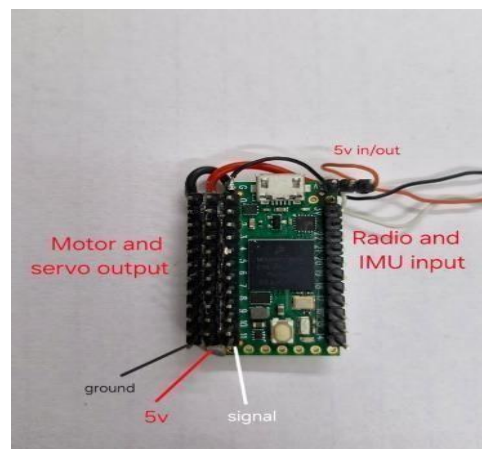


Fig 12 : Hardware Setup of Teensy 4.0

PIN OUTPUTS

The provided pinout configuration offers a comprehensive layout for the connections of a flight controller, highlighting its capabilities for both motor control and peripheral integration. The details are as follows:

Motor Outputs (Pins 0 to 5):

Pins 0 through 5 are allocated for motor connections, labeled as M1 to M6. These outputs utilize the **OneShot125 protocol**, a fast and efficient communication standard that ensures minimal latency in electronic speed controller (ESC) signals. This protocol is particularly advantageous for applications requiring high precision and responsiveness, such as drone flight stabilization and racing.

Servo Outputs (Pins 6 to 12):

Pins 6 through 12 are reserved for servos, identified as S1 to S7. These outputs operate using a **50 Hz PWM (Pulse Width Modulation)** signal, the standard frequency for controlling traditional servo motors. This setup supports a variety of servo-driven mechanisms, such as camera gimbals or landing gear, ensuring compatibility with most off-the-shelf servo units.

Receiver Inputs:

PPM Input (Pin 23):

Pin 23 is designated for PPM (Pulse Position Modulation) receivers, which consolidate multiple control signals into a single line for simplified wiring and reduced interference.

PWM Inputs (Pins 15 to 22):

For legacy receivers or setups requiring individual channel inputs, PWM support is provided as follows:

Ch1 (Throttle): Pin 15

Ch2 (Aileron): Pin 16

Ch3 (Elevator): Pin 17

Ch4 (Rudder): Pin 20

Ch5 (Gear) or SBUS Input: Pin 21

Ch6 (Auxiliary 1): Pin 22

Notably, Pin 21 can also be used for SBUS input, a modern serial communication protocol that consolidates multiple channels into a single wire, reducing complexity and enhancing reliability.

Inertial Measurement Unit (IMU) Connection:

The system includes support for an **MPU6050 IMU** using the I2C communication protocol.

SDA (Data Line): Pin 18

SCL (Clock Line): Pin 19

This IMU provides critical sensor data, including acceleration and gyroscopic information, essential for flight stabilization, orientation control, and autonomous navigation.

This pinout layout reflects a robust and versatile design, ensuring compatibility with a wide range of motors, servos, and receiver types. It also facilitates advanced functionalities, such as high-speed ESC control, flexible receiver options, and precise inertial measurements, making it well-suited for modern drone systems and other UAV applications.

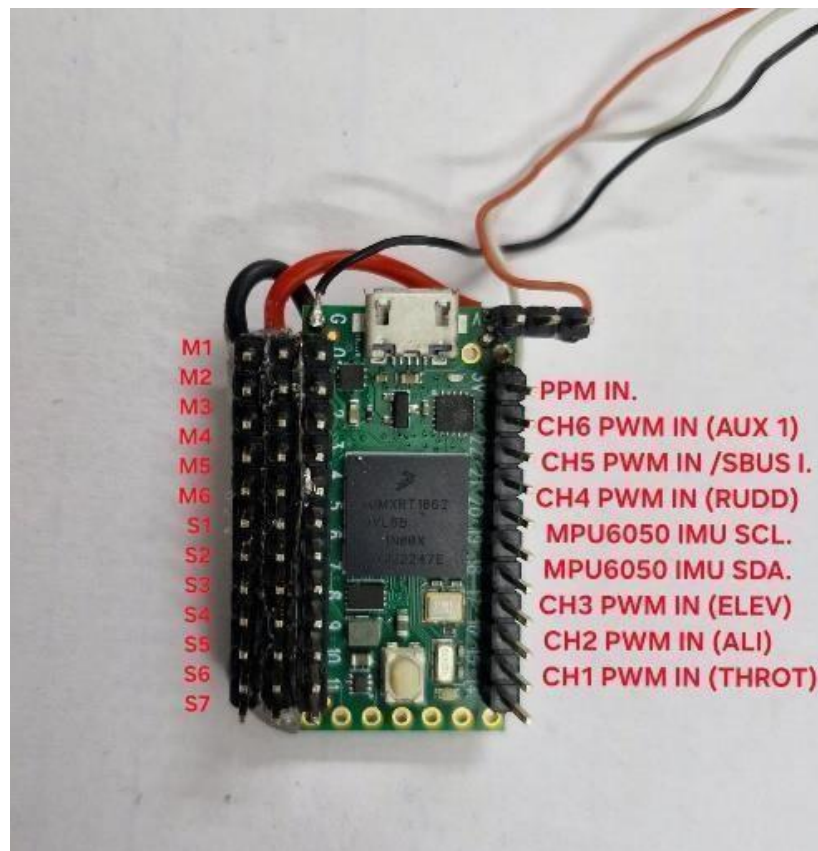


Fig 13 : Pin Outputs of Teensy 4.0

PIN OUTPUT TABLE

OUTPUT	Pin	Protocol
M1	0	OneShot
M2	1	OneShot
M3	2	OneShot
M4	3	OneShot
M5	4	OneShot
M6	5	OneShot
S1	6	PWM
S2	7	PWM
S3	8	PWM
S4	9	PWM
S5	10	PWM
S6	11	PWM
S7	12	PWM
PPM in	23	using PPM receiver
Ch1 (throttle)	15	using PWM receiver
Ch2 (aileron)	16	PWM receiver
Ch3 (elevator)	17	PWM receiver
Ch4 (rudder)	20	PWM receiver
Ch5 SBUS	21	PWM receiver or SBUS
Ch6 P (aux1)	22	If using PWM receiver
IMU SDA	18	MPU6050 IMU I2C connection
IMU SCL	19	MPU6050 IMU I2C connection

5.4 HARDWARE SETUP

1. Flight Controller:

A flight controller serves as the central brain of an unmanned aerial vehicle (UAV), managing various operations to ensure smooth and reliable performance. One of its primary roles is flight stabilization, where it processes real-time data from sensors like gyroscopes and accelerometers to maintain balanced and steady movement by dynamically adjusting motor outputs. It also plays a crucial role in sensor integration, merging inputs from the GPS, inertial measurement unit (IMU), compass, and barometer to accurately determine the UAV's current position, altitude, and orientation. Furthermore, the flight controller handles navigation and autonomous control tasks, which include executing pre-programmed missions such as flying between waypoints, performing automatic take offs and landings, and initiating return-to-home procedures when required. In addition, it facilitates communication between the UAV and the ground control station by transmitting telemetry data, which enables real-time monitoring and allows operators to issue commands remotely. Through these functions, the flight controller ensures that the UAV operates efficiently, safely, and autonomously.

2. Motor (2212 Brushless DC Motor – 1400 kV):

The 2212 brushless DC motor with a 1400 kV rating is a widely preferred choice in UAVs due to its optimal balance of speed, thrust, and efficiency. With a high RPM output, this motor delivers approximately 1400 revolutions per minute per volt supplied, which allows for quick throttle response and stable lift—both critical for achieving smooth and controlled flight. Its compact and lightweight design further enhances its suitability for drone applications, as it reduces the overall payload, thereby improving flight duration, agility, and battery efficiency. The 2212 motor is also known for its durability and low maintenance requirements, thanks to its brushless configuration which minimizes friction and wear. Whether used in hobbyist drones or professional-grade UAVs, this motor provides consistent performance, making it an essential component for stable and efficient aerial operations.



Fig 14: Brushless motor.

3. Drone Frame (QAV 250):

The QAV 250 is a compact, lightweight drone frame specifically designed for high-speed racing and agile flight, making it a popular choice for mini quadcopters. Constructed from strong yet lightweight materials like carbon fiber or composites, it offers excellent durability while keeping the overall weight minimal—an essential factor for enhanced flight performance and extended airtime. With a diagonal motor-to-motor distance of 250 mm, the frame's compact design allows for quick, precise maneuvers, especially in tight spaces or during competitive racing scenarios. Its streamlined structure not only contributes to better aerodynamics but also makes it easier to transport and assemble. The QAV 250 frame provides a perfect balance between sturdiness and agility, making it ideal for drone enthusiasts and racers seeking responsive and reliable performance.



Fig 15 : Drone Frame (QAV 250)

4. Electronic Speed Controller (ESC – 30A, BLHeli Protocol):

Electronic Speed Controllers (ESCs) are essential components in UAVs, particularly quadcopters, where four ESCs are typically used—one for each motor. They play a vital role in controlling the speed and direction of brushless motors to maintain flight stability and maneuverability.

a) 30A Electronic Speed Controller

The 30A ESC is a commonly used variant, capable of handling up to 30 amps of current, which makes it suitable for motors like the 2212 without risk of overheating. These ESCs are designed to operate reliably even during high-thrust or aggressive flight maneuvers. They support a wide voltage range, typically 2S to 4S LiPo batteries, and come equipped with essential protections such as over-current and thermal shutdown to safeguard the system. In quadcopters, a set of four ESCs is required—one per motor—to ensure balanced and synchronized control across all

motors, which is critical for maintaining flight stability.

b) BLHeli Protocol

Many modern ESCs support the BLHeli firmware, which enhances performance by enabling fast and precise motor control. It is compatible with advanced signalling protocols like Oneshot and DShot, which reduce latency and improve responsiveness. BLHeli also allows users to fine-tune various parameters—such as motor timing, braking force, and startup behaviour—using the BLHeliSuite software, allowing for customized and optimized performance tailored to specific UAV requirements. Together, these features make the ESC a vital component in delivering efficient and responsive UAV flight control.

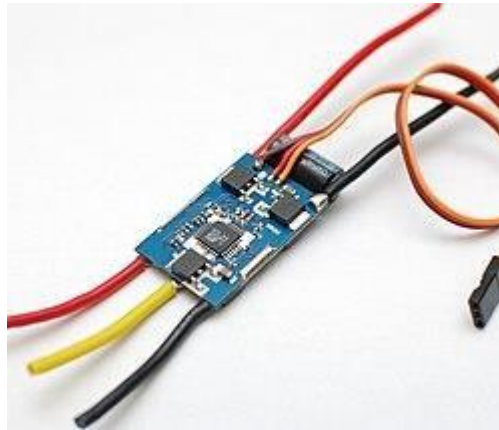


Fig 16: 30A Electronic Speed Controller

5. Power Distribution Board (Matek PDB):

The Matek Power Distribution Board (PDB) is a crucial component in UAV systems, designed to efficiently manage and distribute power from the main battery to all onboard electronics. It serves as a centralized power supply, routing voltage from the LiPo battery to essential components such as electronic speed controllers (ESCs), the flight controller, and peripheral devices. One of its key advantages is its ability to handle high current loads—typically up to 90 amps—making it ideal for drones equipped with 30A ESCs. Additionally, many Matek PDBs come equipped with integrated voltage regulators, such as 5V and 12V BECs (Battery Eliminator Circuits), which are essential for powering sensitive devices like flight controllers, FPV cameras, and video transmitters. Its compact and lightweight design ensures it fits seamlessly into drone frames such as the QAV 250, adding minimal weight while maintaining structural efficiency. Overall, the Matek PDB enhances power management, safety, and reliability in UAV systems.



Fig 17 : **Power Distribution Board (Matek PDB)**

6. FlySky Transmitter and Receiver:

The FlySky transmitter and receiver system is a widely used radio control setup in UAVs, known for its reliability and user-friendly features. Operating on the 2.4 GHz frequency band, it ensures stable, low-latency communication between the drone and the pilot, with strong resistance to signal interference. One of its standout features is multi-channel support—typically ranging from 6 to 10 channels, as seen in models like the FS-i6—which allows comprehensive control over essential flight parameters such as throttle, yaw, pitch, and roll, along with auxiliary functions like flight mode switching or camera operation. This versatility makes it suitable for both beginners and advanced drone pilots. Additionally, the FlySky system is compact, lightweight, and often comes with telemetry capabilities, providing real-time data such as battery voltage and signal strength. Its affordability, dependable performance, and ease of use make it a popular choice in both hobbyist and semi-professional UAV applications.



Fig 18: **FlySky Transmitter and Receive**

7. Battery (4S LiPo Battery):

The 4S LiPo (Lithium Polymer) battery is a widely used power source in UAVs, valued for its combination of high performance and lightweight construction. Delivering a total voltage of 14.8V (3.7V per cell $\times$ 4 cells), it provides ample power to run motors, ESCs, and other onboard electronics, particularly in medium-sized drones. One of its key advantages is its high energy-to-weight ratio and impressive discharge rates—typically ranging from 25C to 60C—ensuring that power is delivered quickly and efficiently during demanding flight maneuvers without adding excessive bulk. Additionally, 4S LiPo batteries are rechargeable and, when handled correctly, offer a long operational lifespan. This makes them not only powerful but also cost-effective for repeated use in UAV missions, contributing to both performance and sustainability in drone operations.



Fig 19: Battery (4S LiPo Battery)

5.5 CIRCUIT DIAGRAM

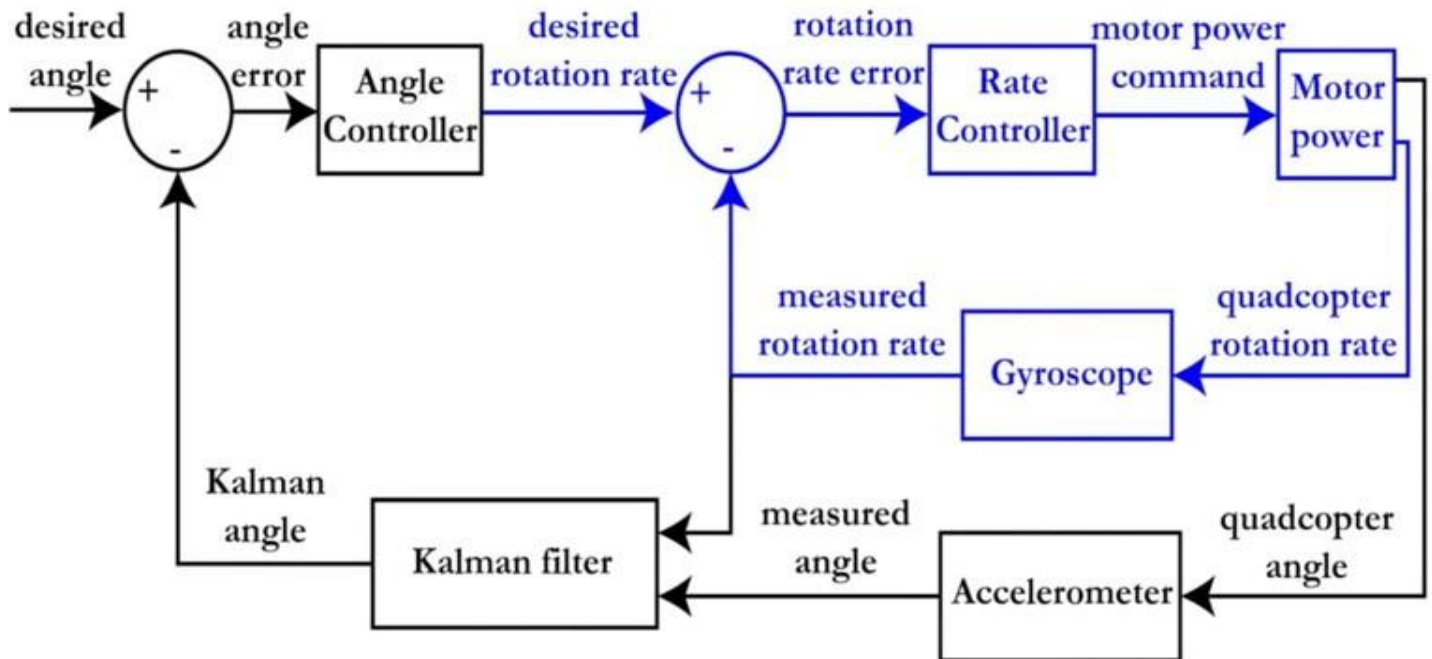
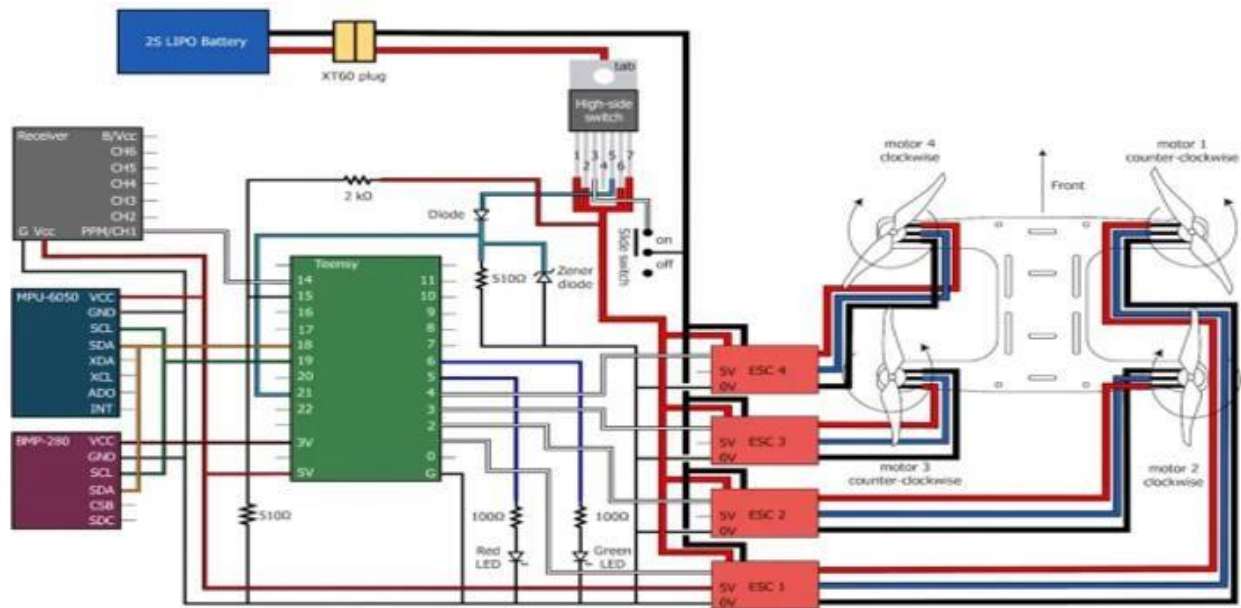


Fig 20: Control Flow Loop

CHAPTER 6

FLIGHT CONTROLLER ALGORITHM

6.1 Structure of the Arduino Sketch

The **Arduino sketch** serves as the main program controlling the flight system. Radio communication functions are separated into a dedicated Radiocom file within the same folder, while all other processes are consolidated in the main sketch.

6.1.1 User-Specified Defines Section

This section is designed for hardware and configuration customization, allowing the user to specify:

- Receiver type (e.g., PPM, SBUS, PWM).
- IMU type (e.g., MPU6050).
- IMU sensitivity values for adjusting the responsiveness of the sensors.

These definitions ensure that the code is compatible with various hardware configurations and application-specific needs.

6.1.2 User-Specified Variable Declaration Section

In this section, critical numerical variables are declared, such as:

- Failsafe values to enhance operational safety during signal loss.
- Controller gains for adjusting the flight controller's responsiveness.

These variables can be easily modified to suit specific requirements without altering the core code.

6.1.3 Pin Declaration Section

This section assigns and declares hardware pin connections, such as motor outputs, receiver inputs, and IMU communication lines. All pin assignments are accompanied by comments for clarity, making it straightforward to understand and adapt the configuration.

6.1.4 Setup Section (`void setup`)

The `setup` function initializes the flight system and is executed once at startup. Key tasks include:

- Setting up communication protocols like I2C for the IMU and Serial for debugging.
- Running calibration routines, such as initializing the IMU and electronic speed controllers (ESCs).
- Configuring failsafe mechanisms to handle system errors safely.

6.1.5 Loop Section (`void loop`)

The `loop` function contains the flight control code and is executed continuously after the setup phase. This section handles real-time flight operations, including:

- Reading inputs from the receiver and IMU.
- Processing sensor data for stabilization and navigation.
- Sending commands to ESCs and servos for motor and control surface actuation.

6.1.6 Critical Functions

The sketch concludes with all critical functions that are called in the `setup` and `loop` sections. Each function is designed to perform a specific task, such as sensor data acquisition, PID control calculations, or actuator commands. Functions are segmented and named to ensure readability and ease of understanding. This modular structure enables efficient debugging, flexibility for hardware modifications, and scalability for integrating additional features.

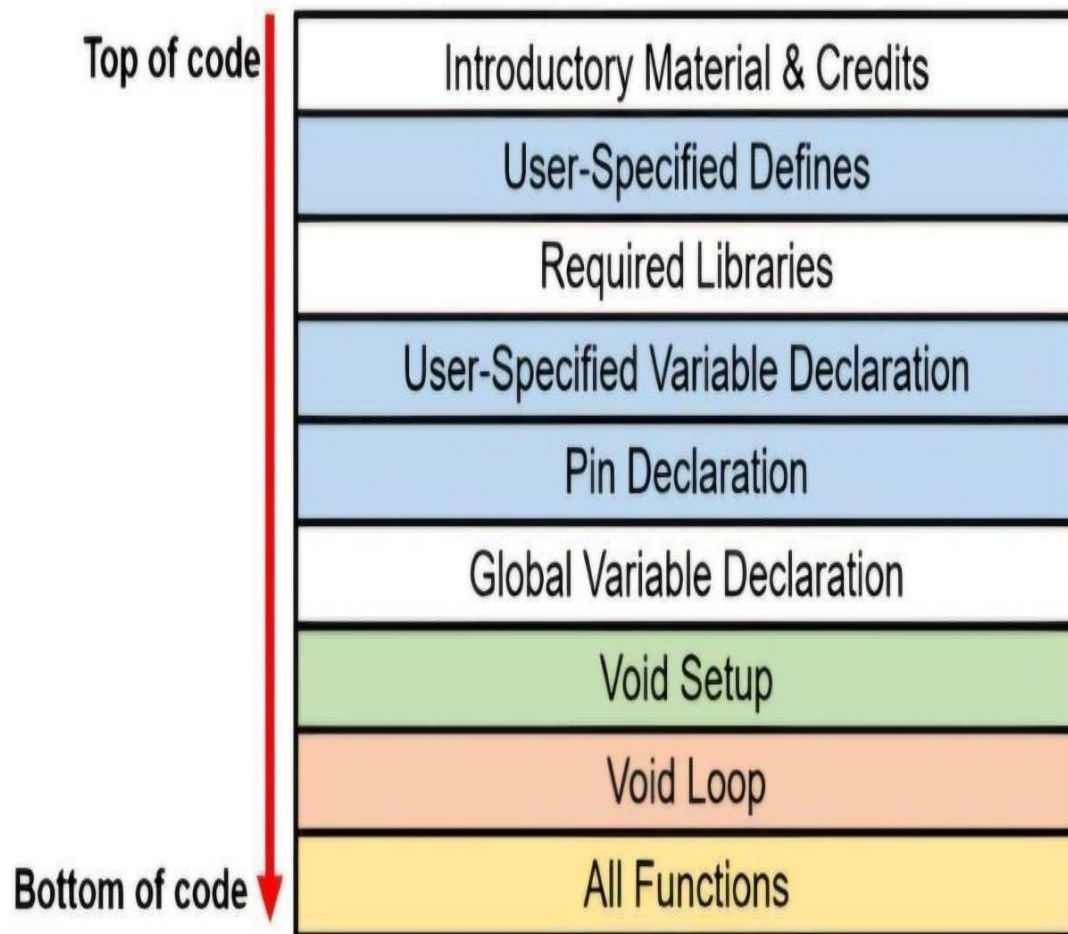


Fig 21: General overview of a flight control loop allowing stabilization of a small aerial vehicle

6.2 WORKFLOW OF FLIGHT CONTROLLER ALGORITHM

The workflow of the flight controller code begins with system initialization, where hardware components like the IMU, ESCs, and receiver are set up, calibrated, and configured for communication. Once initialized, the code transitions to a continuous loop where it reads sensor data from the IMU and receiver inputs, processes the data to calculate errors between desired and actual states, and applies PID control algorithms to determine corrective actions. These calculated outputs are then mixed with user commands and sent to the motors and servos for real-time stabilization and control. Throughout the process, failsafe mechanisms monitor for signal loss or errors to ensure safe operation.

6.2.1 Initialization and Setup Functions

void setup(): This is the primary initialization function that is executed once upon startup. It prepares all hardware and software components for operation. Key tasks include: Configuring communication protocols (I2C for IMU, Serial for debugging or telemetry) Initializing input devices like receivers (PPM, PWM, or SBUS). Running calibration routines for the IMU. Initializing output devices like ESCs and servos.

void initializeIMU(): Configures the IMU (e.g., MPU6050) for communication over I2C. Sets sensitivity levels for gyroscopes and accelerometers. Performs a calibration routine to establish baseline values for accurate motion sensing.

void initializeESCs(): Sends startup signals to the ESCs (Electronic Speed Controllers). Ensures ESCs are ready to receive commands (e.g., OneShot125 initialization or standard PWM).

void initializeReceiver(): Detects the type of receiver being used (PPM, PWM, or SBUS). Configures the corresponding input pins for reading control signals. Calibrates the receiver input range (minimum, maximum, and neutral positions).

void initializeFailsafe(): Configures failsafe mechanisms to handle signal loss or critical errors. Sets default motor states or triggers safety measures like disarming the motors.

1. Sensor Data Acquisition Functions

void readIMU(): Reads raw gyroscope and accelerometer data from the IMU. Applies calibration offsets to correct sensor drift. Computes processed values such as pitch, roll, and yaw angles using complementary or Kalman filters.

void readReceiverInputs(): Captures input signals from the receiver (e.g., throttle, aileron, elevator, rudder, and auxiliary channels). Maps input values to a standard range for processing (e.g., 1000–2000 μ s to 0–100%).

2. Control Functions

void calculatePID(): Implements PID (Proportional-Integral-Derivative) control for pitch, roll, and yaw stabilization. Computes corrective outputs for motors based on the error between desired and actual states.

void mixMotorOutputs(): Combines PID control outputs with user inputs (e.g., throttle, roll) to generate individual motor commands. Ensures the output is within acceptable ranges for the ESCs.

void processFailsafe(): Detects signal loss or abnormal sensor values. Sets motors to a predefined safe state (e.g., idle throttle or full disarm).

3. Output Functions

void writeMotorOutputs(): Sends calculated signals to the ESCs via the specified protocol (e.g., OneShot125 or PWM). Maps motor values to the required range for the ESCs (e.g., 1000–2000 μ s for PWM).

void controlServos(): Sends position commands to servos for auxiliary functions, such as gimbal stabilization or landing gear operation.

4. Debugging and Telemetry Functions

void sendTelemetry(): Sends flight data (e.g., IMU readings, PID outputs, receiver inputs) over Serial or other communication interfaces for real-time monitoring.

void logData(): Stores critical flight data in memory for post-flight analysis. Useful for debugging or performance optimization.

void displayDebugInfo(): Outputs debug messages to the Serial Monitor. Provides insights into system state, such as initialization status, sensor values, or control outputs.

5. Main Control Loop Functions

void loop(): The primary control loop that runs continuously after **setup()**.

It performs the following tasks in sequence:

Reads sensor data from the IMU.

Captures user inputs from the receiver.

Processes sensor data and calculates stabilization controls using PID.

Mixes user commands with stabilization outputs.

Writes motor and servo outputs and Monitors failsafe condition

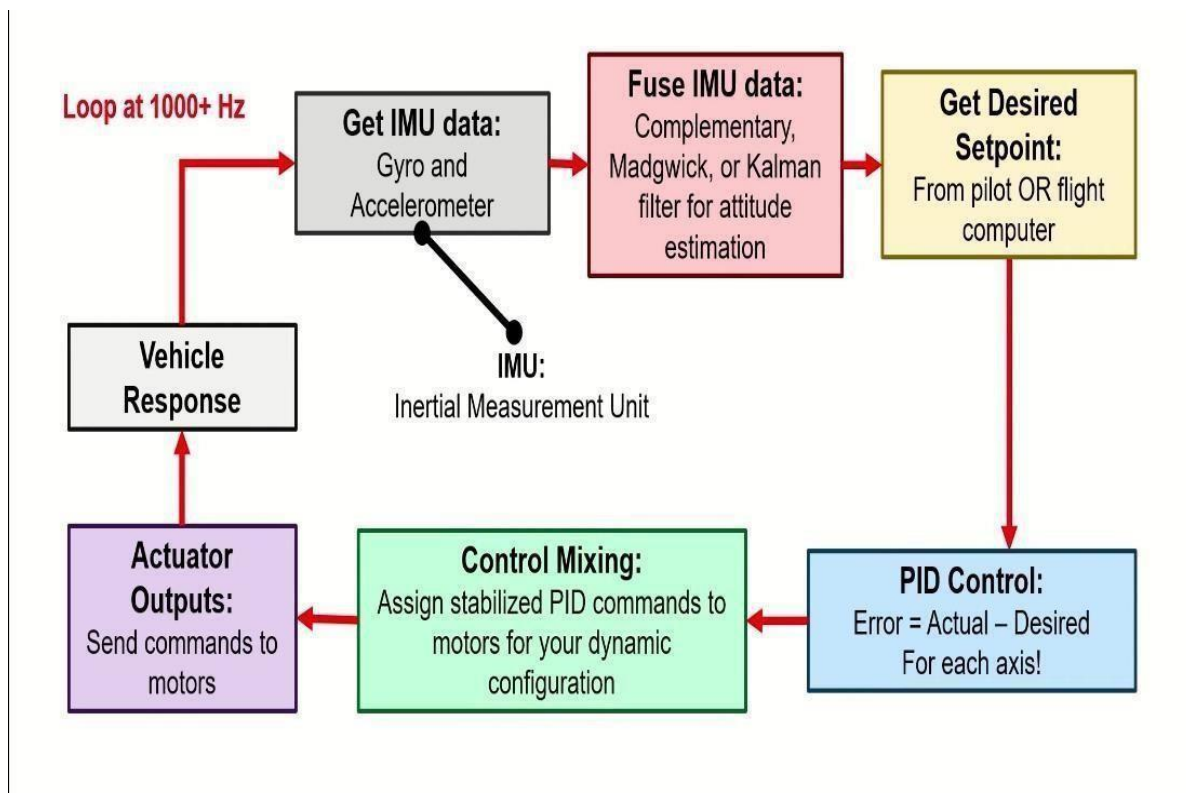


Fig 22 : Workflow of Flight Control System

6.2.2 VOID SETUP()

The void setup part of the code runs just once when you power on the flight controller. Its main job is to get everything ready—like setting up communication with sensors, pins, and the radio. It also handles important safety checks, such as making sure the failsafe is active before any motors are armed. This way, the system is fully prepared and safe before starting the main flight loop. You can find more details about what each function does and which variables are used in the Functions List below.

TABLE 1 : List of functions in void setup and there function

Step	Function/Process	Description
1	Serial.begin()	Establish serial connection to onboard USB
2	Initialize all pins	Motor and servo pins are assigned/attached to their designated pins from the pin declaration section
3	radioSetup()	Initialize communication with desired radio type using function within radioComm sketch
4	Failsafe setup	Set radio channel variables to default values before entering main loop
5	IMUinit()	Initialize communication with desired IMU
6	calculate_IMU_error()	Get error offset from IMU gyro and accelerometer to apply to readings in the main loop
7	Arm servo/PWM channels	Write low value to servos/motors using PWM
8	Arm OneShot125 channels	Write low value to ESCs using OneShot125
9	calibrateAttitude()	Warms up IMU filter before entering main loop

6.2.3 VOID LOOP()

The void loop section is where the main flight control code runs continuously. It's made up of several smaller functions, and each one handles a specific part of the flight control process. These functions are called one after another in every loop cycle, working together to keep the aircraft stable and responsive. A full breakdown of what each function does, the variables it uses, and other important details is available in the Functions List below.

Table 2 : List of functions in void loop and there function

Step	Function/Process	Description
1	Print Data (commented out default)	Multiple print statements used for troubleshooting; prints data to serial monitor at 100 Hz
2	getIMUdata()	Pulls raw gyro, accelerometer, and magnetometer data from IMU and low-pass filters to remove noise
3	Madgwick()	Uses IMU data to estimate absolute vehicle attitude
4	getDesState()	Convert raw radio commands to normalized values based on saturated limits (max rate or max angle in user-specified variables section)
5	Controller	Option of controlANGLE(), controlANGLE2(), or controlRATE() PID control to generate stabilized axis variables based on estimated attitude and desired state; gains adjusted in user-specified variables section
6	controlMixer()	Mixes PID controller outputs to scaled actuator commands based on dynamic configuration
7	scaleCommands()	Scale motor/actuator commands to required range for writing out to designated pins (OneShot125 default)
8	throttleCut()	Directly set motor commands to low/off value if throttle cut switch is engaged
9	commandMotors(), Command servos	Send ESC commands to OneShot125 pins and write servo commands to PWM pins
10	getCommands()	Get current radio commands to be used in next loop iteration
11	failsafe()	Check if received radio commands are valid and default to failsafe values if they are not
12	loopRate()	Idle in main loop until specified loop frequency is achieved (2 kHz default)

6.2.4 FUNCTIONS LIST

This section gives a detailed description of every function contained within the code

RadioSetup(): This function sets up communication between the flight controller and your radio receiver, based on what type of receiver you've chosen in the settings. For receivers that use PWM, the channel pins (chXPin) are set up to trigger an interrupt whenever their signal changes. This lets the program immediately read that channel's signal, no matter what else the controller is doing at the time. These readings capture how long each signal is high, which tells us the command the receiver is sending—usually somewhere between 1000 to 2000 microseconds. The latest values for each channel are saved and used later in the `getCommands()` function whenever needed.

IMUinit() This function starts up the IMU (the motion sensor), which is usually the MPU6050 connected via I2C at address 0x6B. If you've selected an MPU9250 instead, it sets that up using SPI. The code assumes your IMU is connected to pins 18 (SDA) and 19 (SCL) on the board.

Calculate_IMU_error(): When you power on the board, this function reads the IMU's accelerometer and gyroscope values 12,000 times and calculates an average to figure out the sensor's natural offset (or error) while it's still and level. These offset values are later subtracted from the live readings to make sure the data starts from zero. The only exception is the Z-axis on the accelerometer, which should read 1 because of gravity. These calibration numbers are also shown in the serial monitor

calibrateAttitude(): Before the flight loop starts, this function runs a mini loop for 5 seconds (10,000 times at 2000 Hz) to "warm up" the IMU and Madgwick filter. It helps the system lock in on its starting orientation using just a few key functions: `getIMUdata()`, `Madgwick()`, and `loopRate()`.

getIMUdata(): This function grabs the current accelerometer, gyroscope, and (if available) magnetometer readings from the IMU. It applies a basic low-pass filter to reduce high-frequency noise. The data is then converted into meaningful units: G-forces, degrees per second, and microtesla for the magnetic field. The filtering behavior is controlled by variables you can adjust in the code, helping you balance between smooth data and responsive performance.

Madgwick(): This function combines the IMU data (acceleration, rotation, and magnetic field) to estimate the orientation of the aircraft—basically its roll, pitch, and yaw angles. It uses the Madgwick algorithm, which works with quaternions for efficient and accurate angle tracking. If the magnetometer isn't available (like with the MPU6050), it switches to a 6-axis version of the algorithm instead. A tuning variable called `B_madgwick` lets you adjust how much influence the accelerometer and gyroscope have on the results.

getDesState() This function takes the incoming PWM signals from your radio and converts

them into usable commands like throttle (thro_des), desired roll, pitch, and yaw. Throttle is always scaled between 0 and 1, while the others are scaled based on the limits you've set in the code (either in degrees or degrees per second, depending on the flight mode). It also creates "passthrough" variables for roll, pitch, and yaw, which you can use to send raw, unstabilized commands straight to motors or servos if needed.

controlRATE(): This is the part of the code where stabilization happens. It uses a simple PID controller to keep the aircraft stable based on the difference between what you want it to do (from the radio) and what it's actually doing (from the gyroscope). The controller calculates how much correction is needed for each axis—roll, pitch, and yaw—and outputs those corrections as roll_PID, pitch_PID, and yaw_PID. You can tune how strong or gentle these corrections are by adjusting the P, I, and D values in your code. No matter what control mode you're using, the yaw axis is always stabilized using this method.

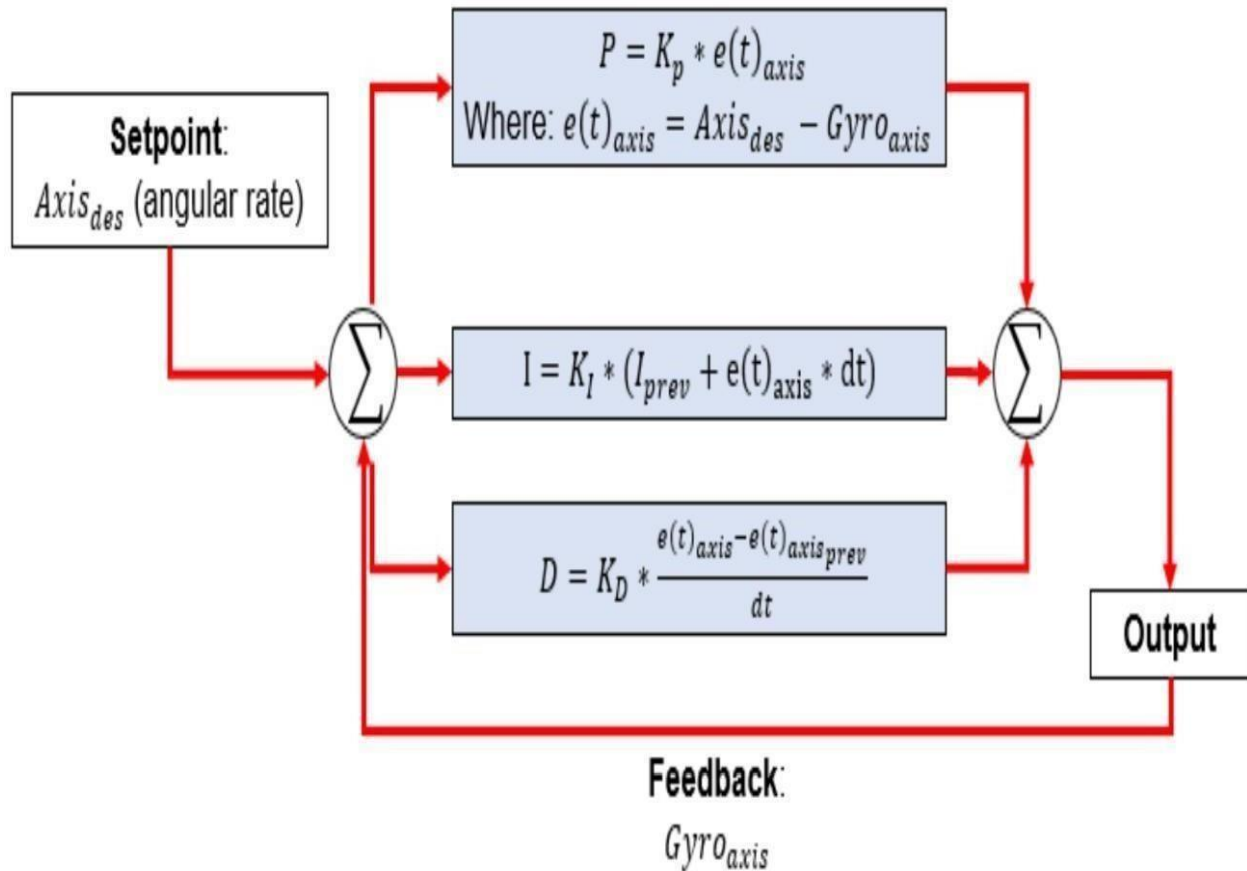


Fig 23: PID controller stabilizing on angular rate setpoint.

Control ANGLE(): This function uses a basic PID controller to stabilize the aircraft based on its angles (not rates). It calculates how much correction is needed by comparing the desired roll and pitch angles (from the radio) with the actual roll and pitch angles (measured by the IMU). The result is used to generate the PID output values: roll_PID and pitch_PID. You can fine-tune how responsive or smooth the control feels by adjusting the P, I, and D gain values for each axis in the settings section of the code. Keep in mind, even when you're using angle mode for roll and pitch, the yaw axis is always controlled using rate mode (through controlRATE()).

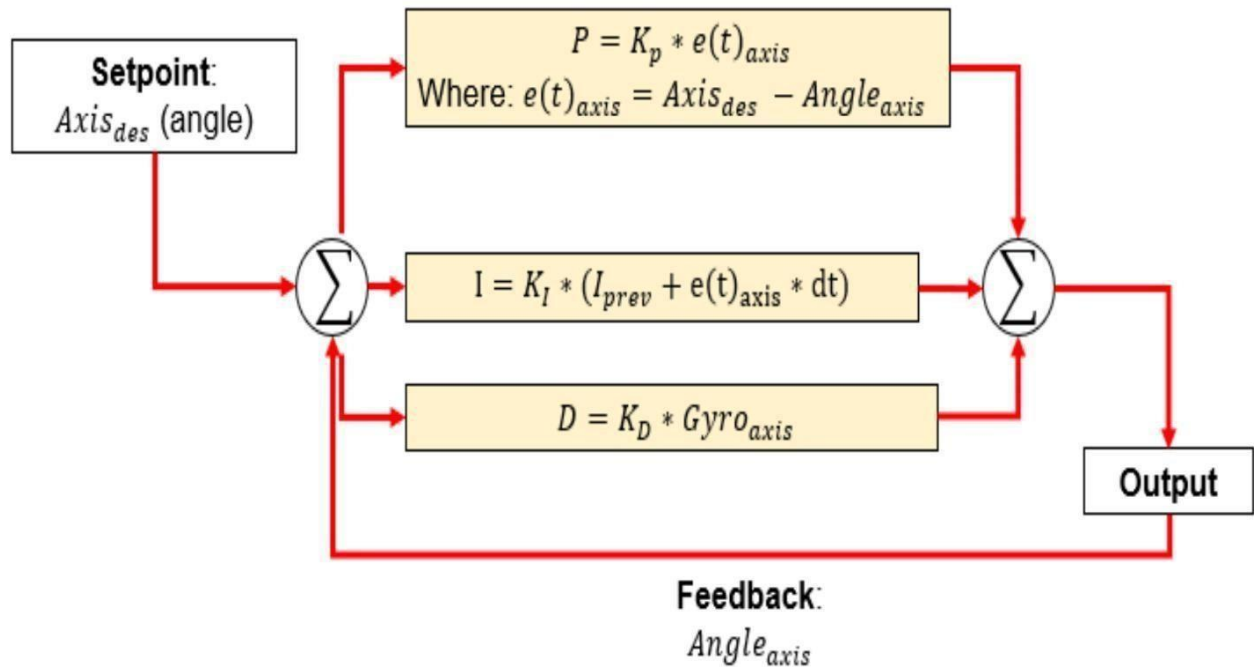


Fig 24: PID controller stabilizing on angle setpoint

Control Mixer (): This function combines the stabilized control outputs from the PID controller—roll_PID, pitch_PID, and yaw_PID—along with the throttle input (thro_des) to create actuator control signals. These are called mX_command_scaled for motors and sX_command_scaled for servos, and they're kept in a normalized range of around 0 to 1. The default setup mixes the signals for a basic quadcopter (four motors). If you're using servos (like in airplanes), the same PID outputs can be used for them too. Also, if you want to send direct commands from your transmitter instead of using stabilization, you can use roll_passthru, pitch_passthru, and yaw_passthru. Once everything is mixed and scaled, another function—scaleCommands()—adjusts the values to match the actual signal range expected by ESCs or servos.

scale Commands (): This function takes the normalized values from mX_command_scaled and sX_command_scaled and converts them into actual signal values (mX_command_PWM and sX_command_PWM) that the hardware can understand. For example, if you're using OneShot125 for ESCs, the command range is scaled between 125 to 250 microseconds. For servos, it's usually 0 to 180 degrees. It also makes sure the final values stay within safe limits so the motors and servos don't get bad signals.

commandMotors(): This function sends the final PWM signals to the ESCs using the OneShot125

protocol. It uses the `mX_command_PWM` values to decide how long each motor signal pin should stay high. All motor pins are turned on (set high) at once, and then the code checks each one to turn it off (set low) at the right time. Once all the motor pins are turned off, the loop ends.

calibrateESCs(): Use this function to calibrate your ESCs. It acts like a simplified version of the main loop, where your throttle input goes directly to the motor outputs. To use it, just uncomment the `calibrateESCs()` function in `void setup()`, upload the code, and power the flight controller with your throttle stick at max. The ESCs will make their calibration beeps. Then bring the throttle to the lowest position and your ESCs will complete the calibration.

CHAPTER 7

RESULTS OF MATLAB SIMULATION

The graph in the image displays the X, Y, and Z coordinates of the quadrotor as a function of time. Each line represents the motion in one of the axes:

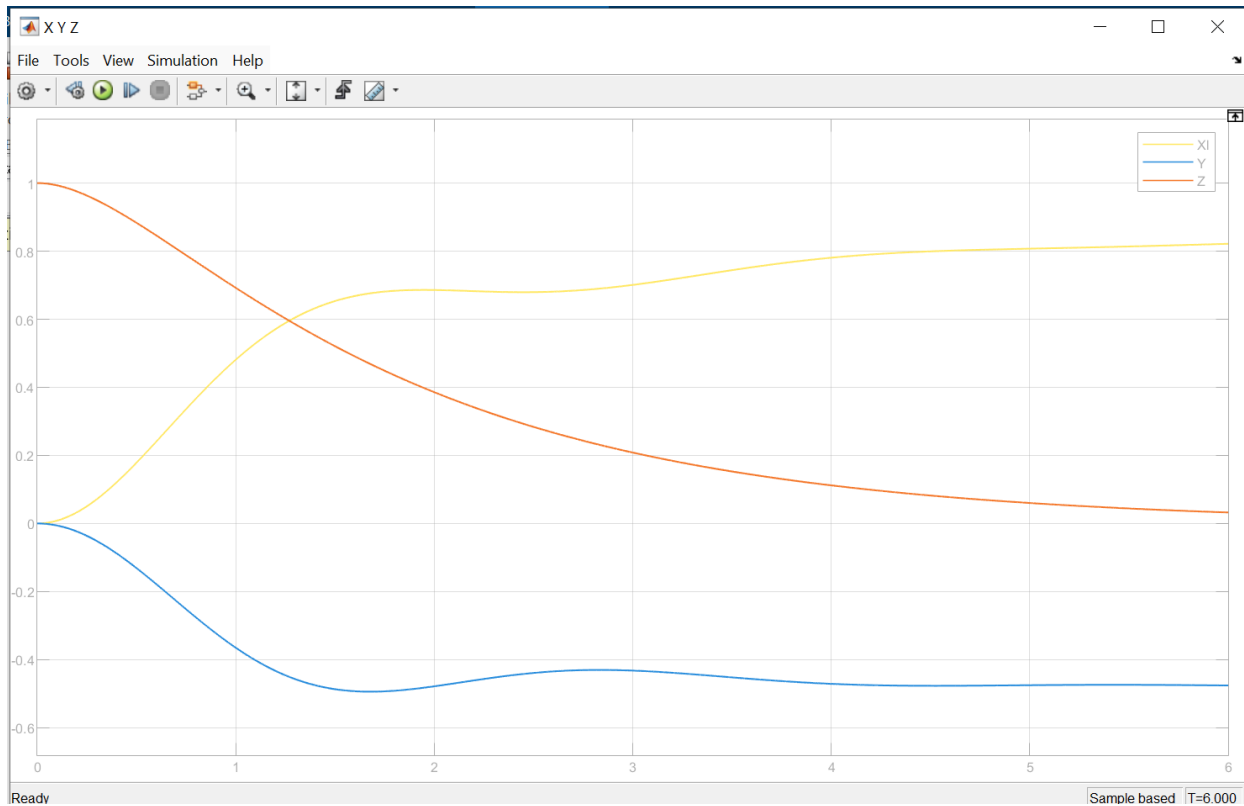


Fig 25 : XYZ Plot

The graph represent the angular velocity (rad/s) of four propellers over time. At the start of the simulation, the propeller speeds exhibit sharp changes, which are characteristic of a system responding to initial conditions or control commands.

Oscillatory Behavior : There is an oscillatory nature in the curves, especially evident in the early seconds, indicating the system is trying to stabilize. This due to PID feedback controllers compensating for disturbances or setpoint changes.

Steady-State : After around 2-3 seconds, the propeller speeds converge to a relatively stable value. This suggests that the system achieves equilibrium, maintaining stable flight or orientation.

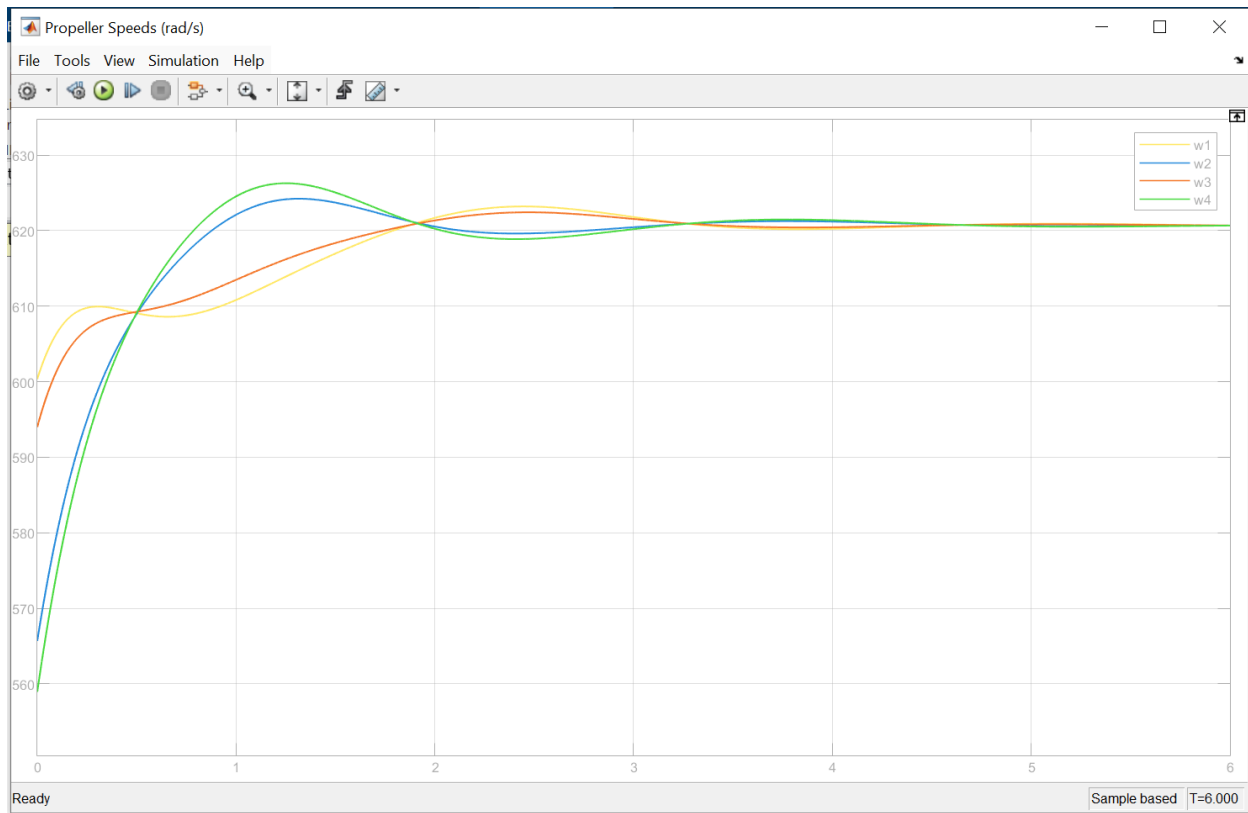


Fig 26 : Propellor speed vs time plot

This graph represents Roll, Pitch, and Yaw angles over time, which are essential components of a quad-rotor system's attitude and position control. This shows that the control system is responding effectively to disturbances or setpoints, stabilizing the angles over time.

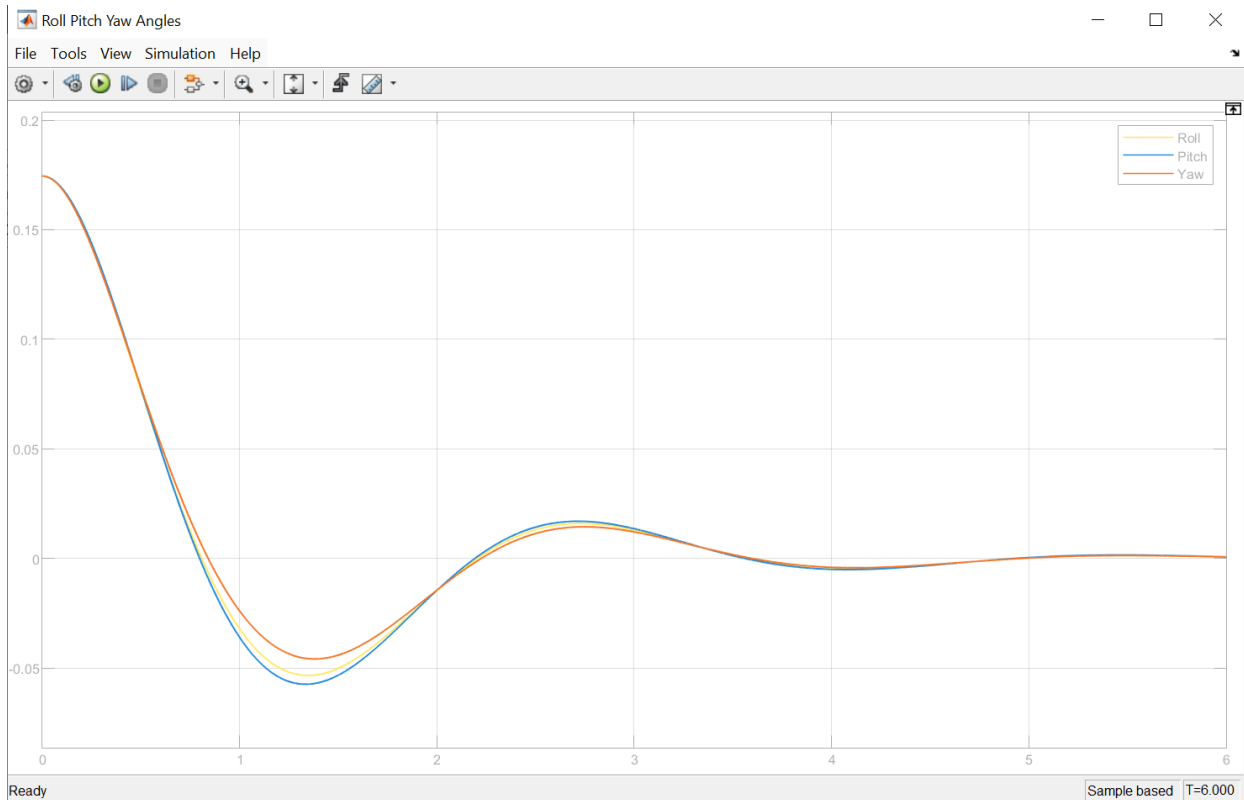


Fig 27: Roll pitch yaw angles vs time plot

This plot displays the torque responses (denoted by τ) for Roll, Pitch, and Yaw as a function of time in a quadrotor simulation. These torques are essential for controlling the orientation of the quadrotor. After the initial overshoot, the torques oscillate and then gradually converge to zero. This indicates that the controller is able to damp the oscillations and stabilize the quadrotor orientation over time. In the final part of the graph, all three torques approach zero. This steady-state behavior confirms that the controller successfully eliminates the orientation error.

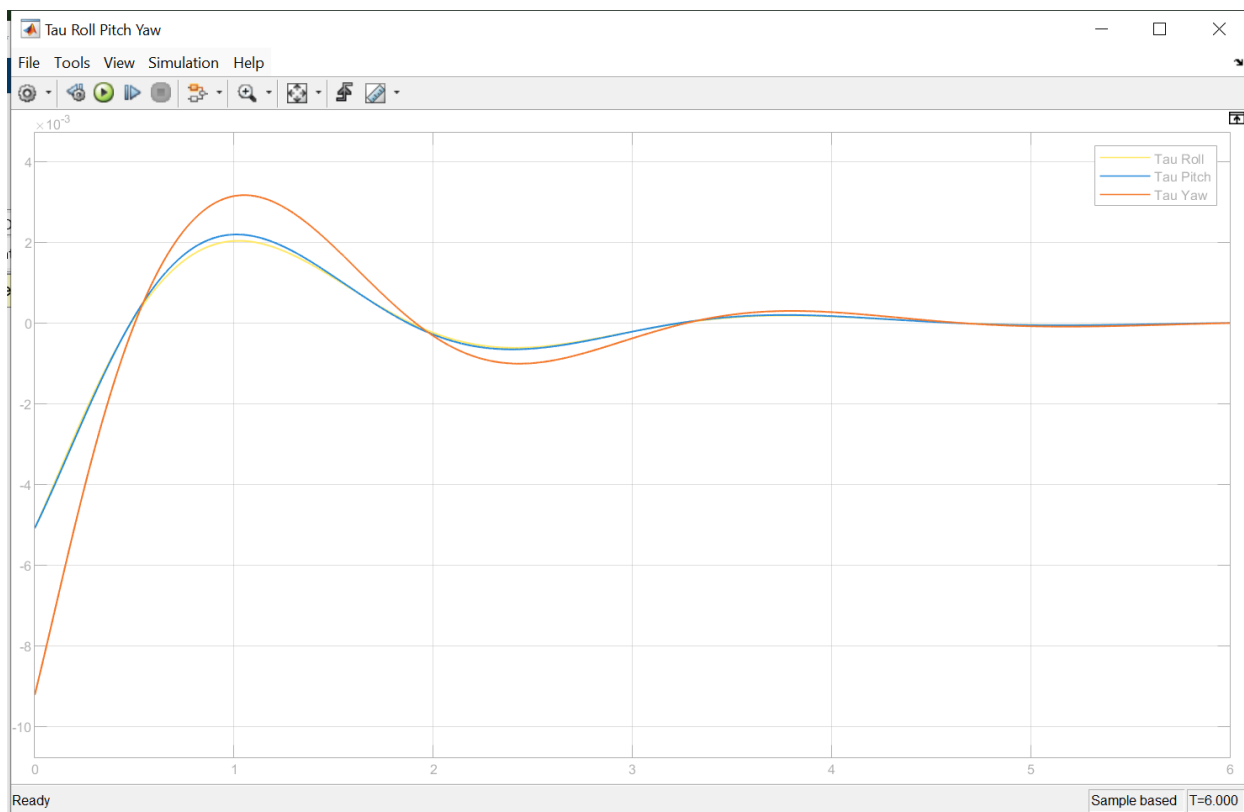


Fig 28: Torque roll, pitch, yaw d vs time plot

7.1 FLIGHT CONTROLLER ALGORITHM RESULT

The following plot shows **printRadioData()** function is designed to debug and monitor the input signals received from the drone's radio receiver which Outputs raw pulse-width modulation (PWM) or processed values from the receiver

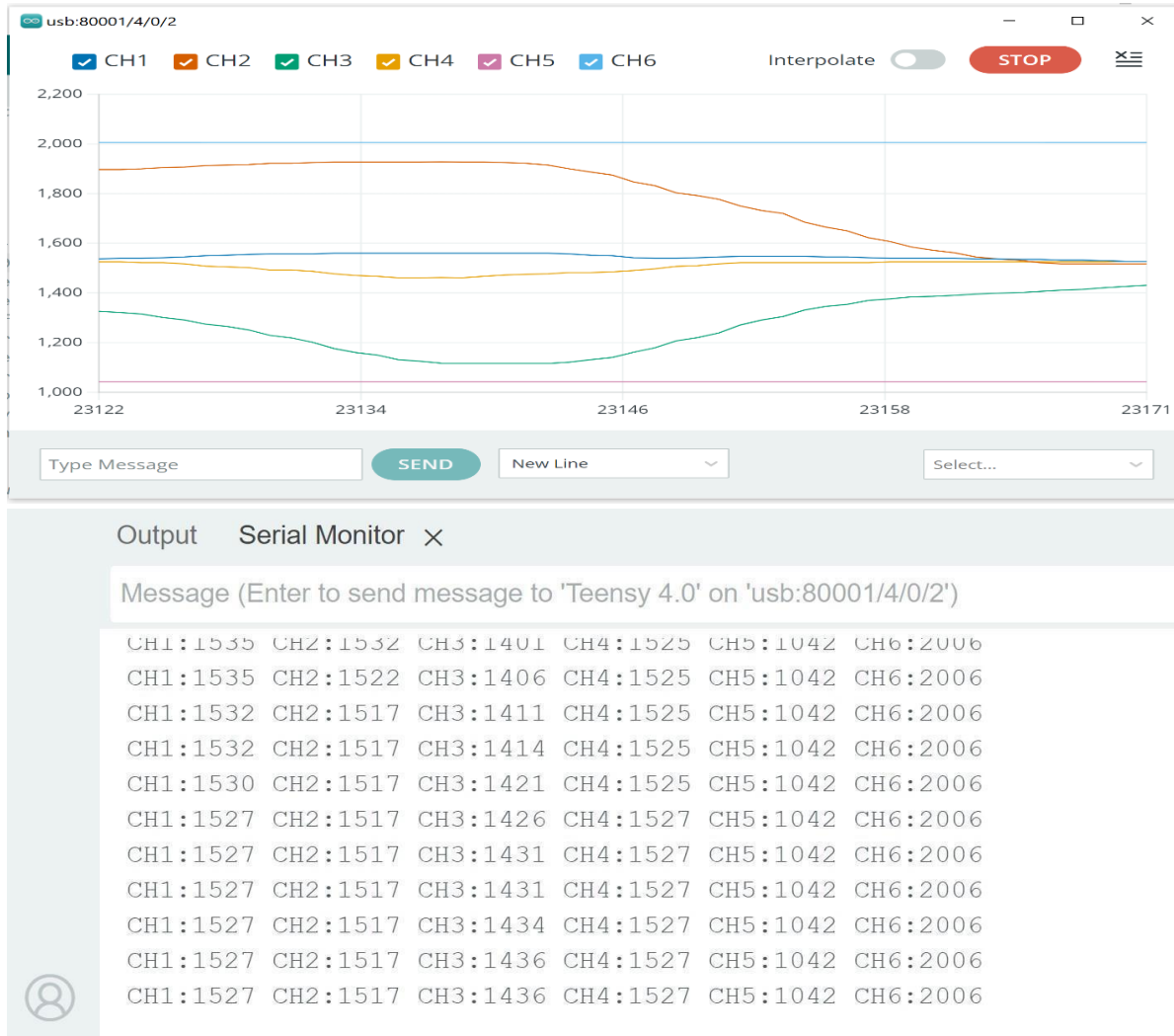


Fig 29 : Receiver input data

The following plot shows **printGyroData()** function is designed to display real-time gyroscope readings from the IMU (Inertial Measurement Unit). It helps verify the accuracy of angular velocity measurements (pitch, roll, yaw) and ensures that the IMU is functioning correctly. Displays the raw gyroscope data (angular velocities in X, Y, and Z axes) read from the IMU.

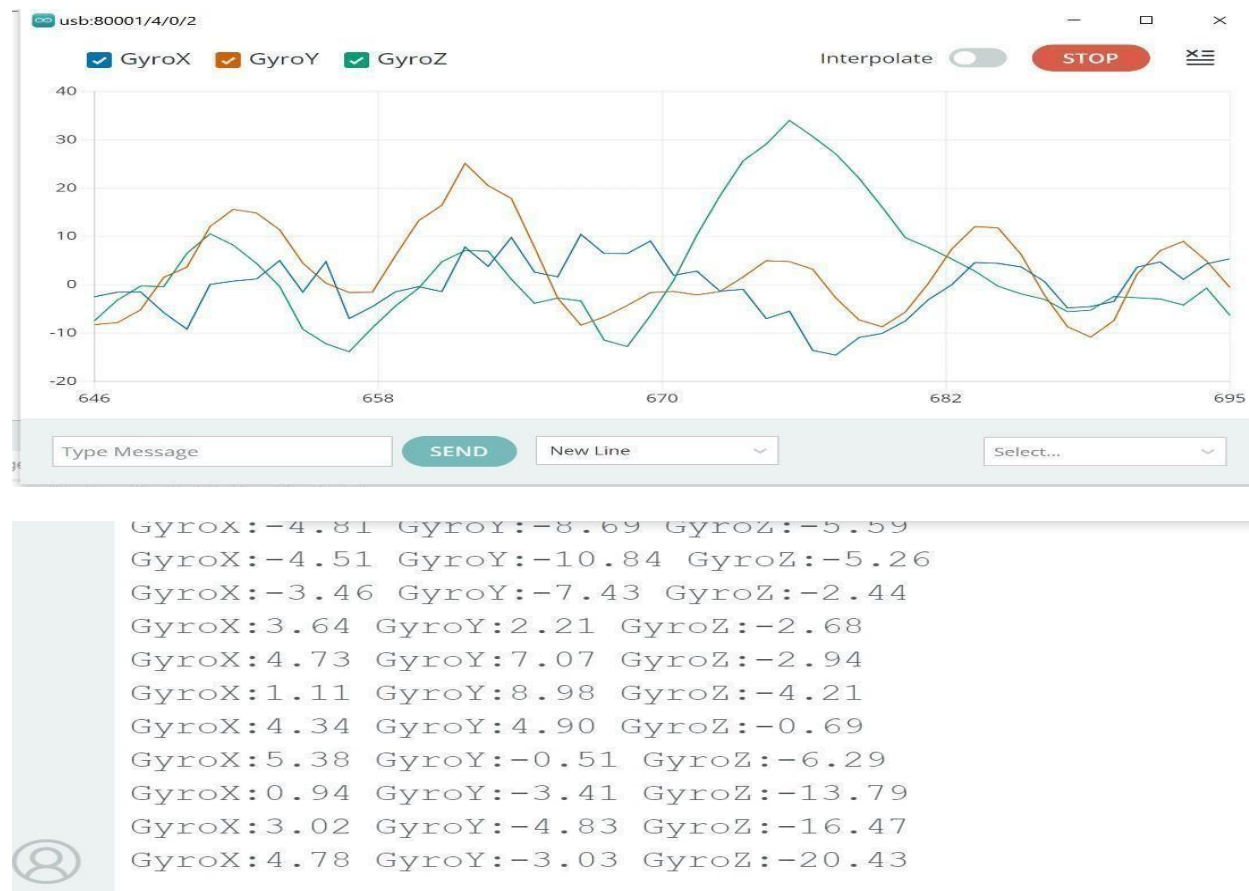


Fig 30 : Gyro input data

The following **printAccelData()** function is used to output real-time accelerometer data from the IMU, showcasing the drone's acceleration along the X, Y, and Z axes.

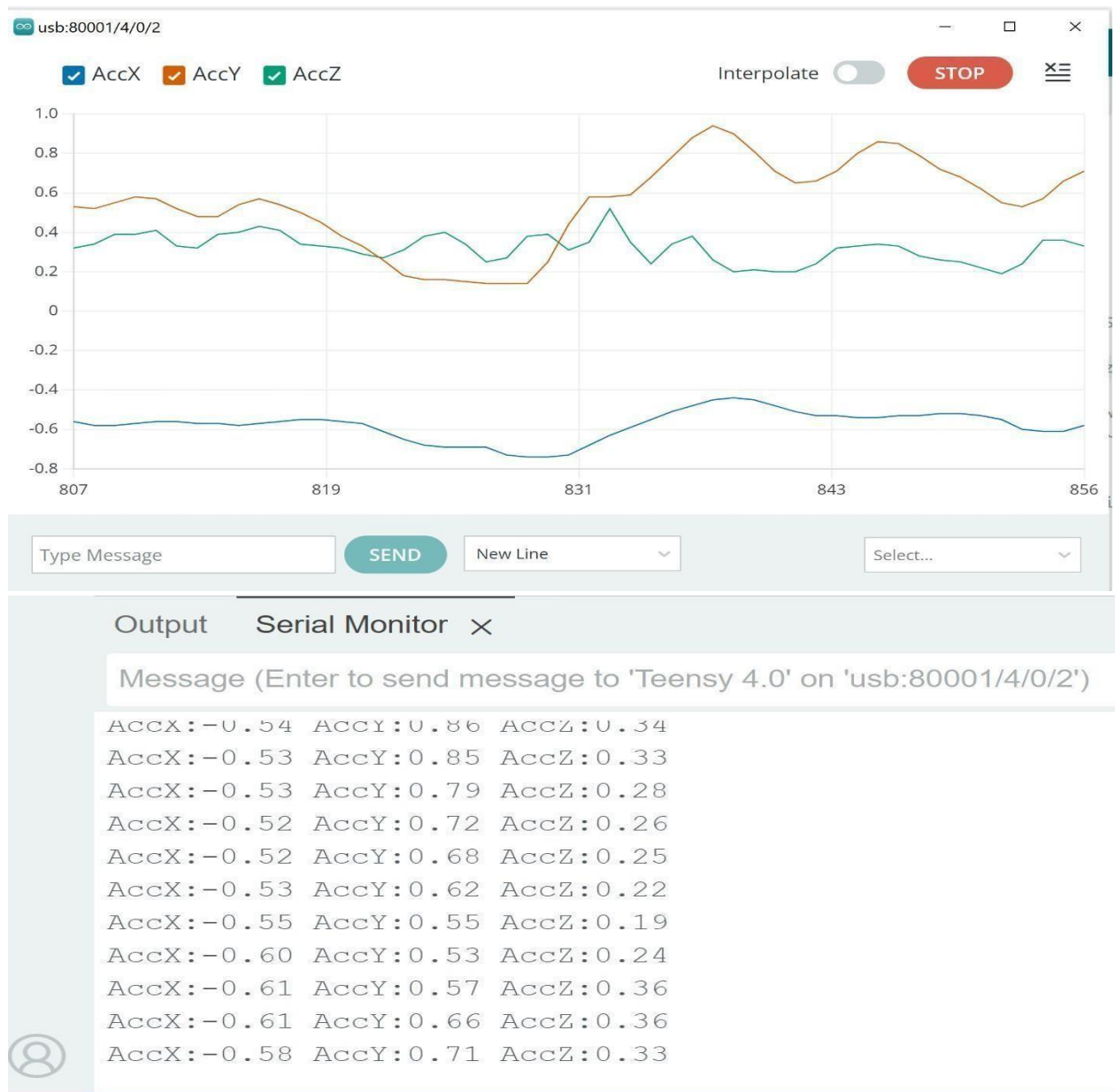


Fig 31: Acceleration input data

The following **getDesiredState()** function determines the desired state of the drone based on user inputs received from the transmitter or controller. This state includes parameters like throttle, roll, pitch, and yaw, which are essential for guiding the drone's movement and attitude.

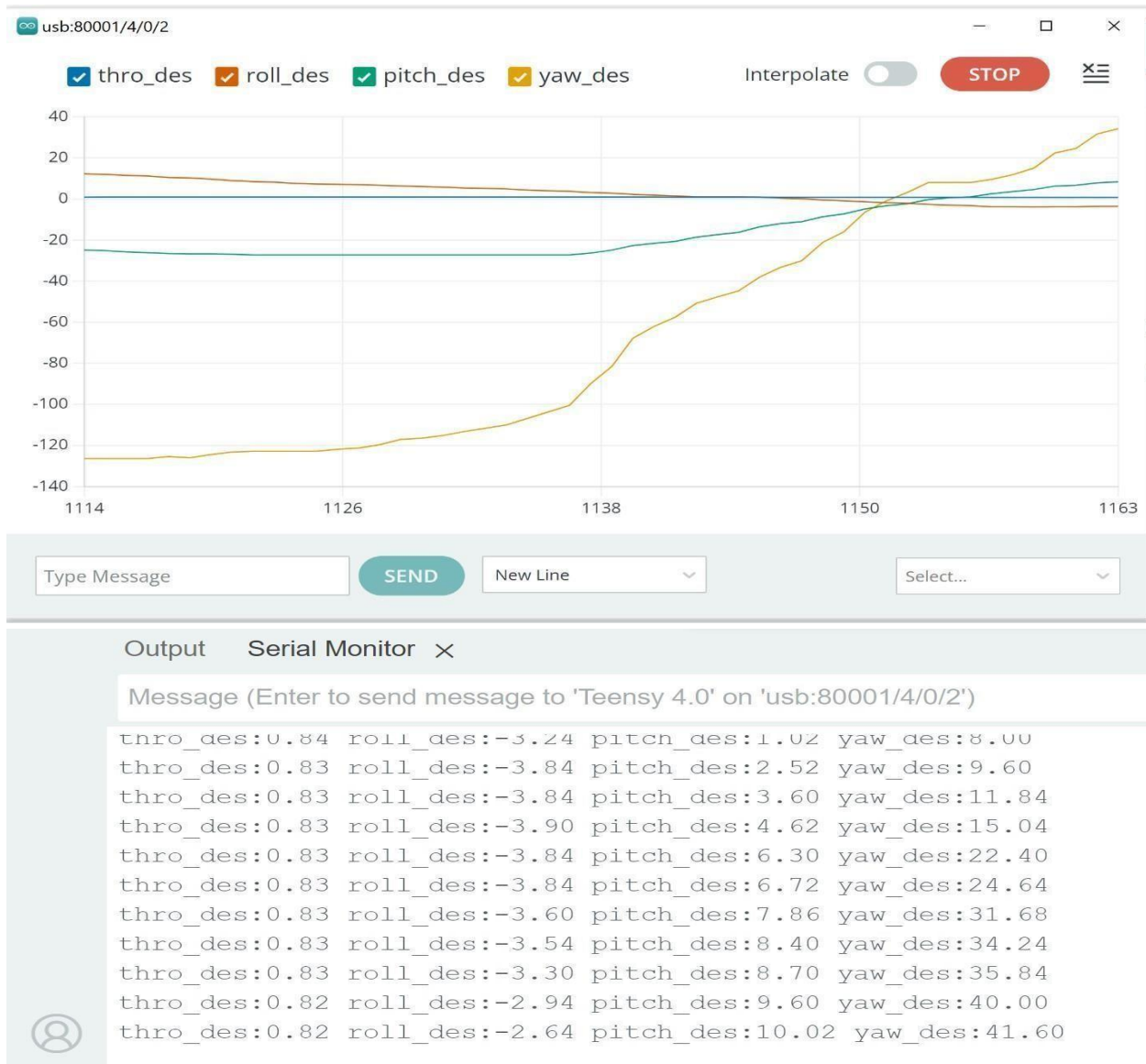


Fig 32 : Desired state output data

The following **printRollPitchYaw()** function is responsible for displaying the current orientation of the drone in terms of roll, pitch, and yaw angles. These angles are calculated based on sensor data from the IMU and provide critical insight into the drone's attitude during flight

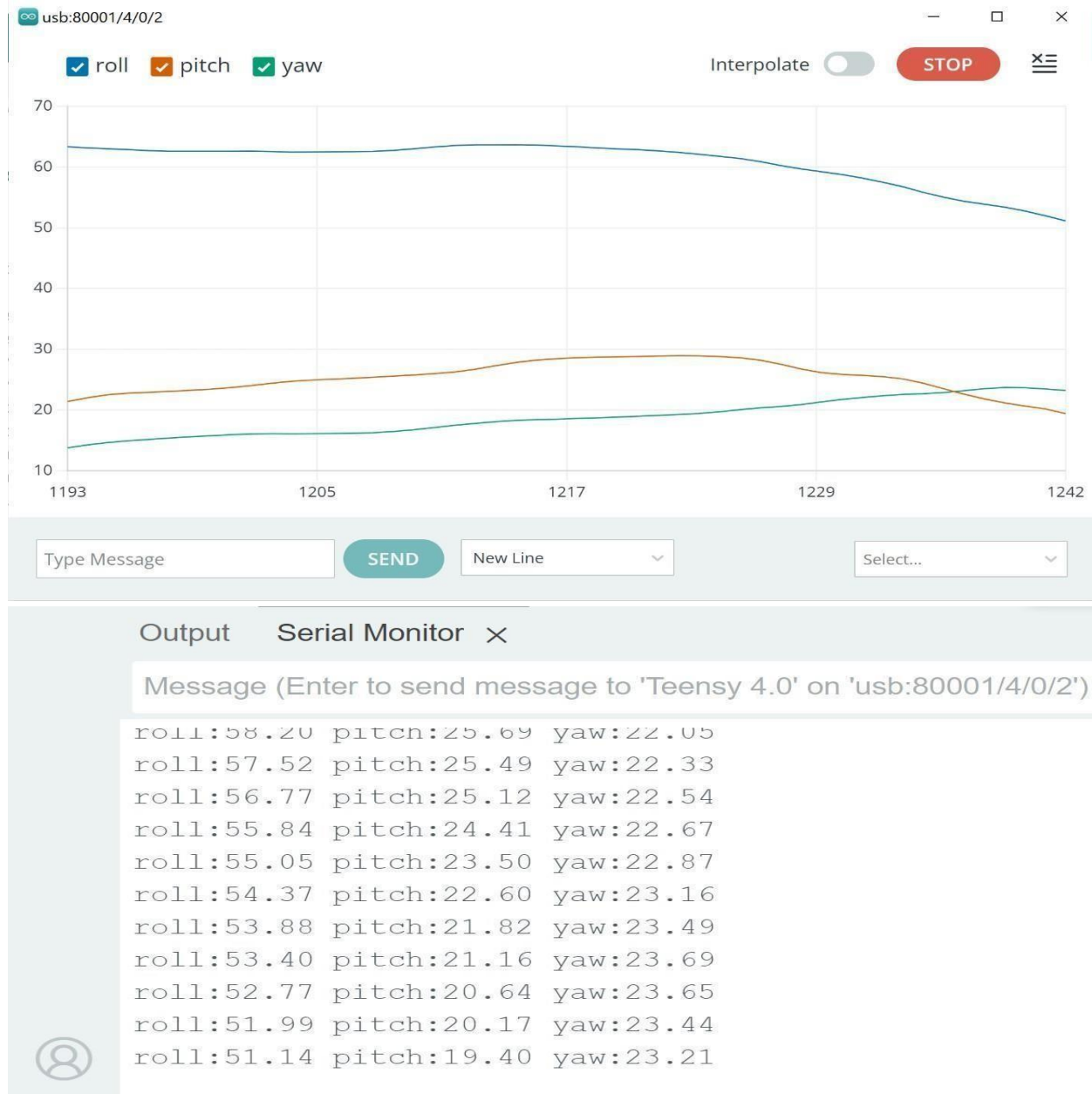


Fig 33: Roll pitch yaw angle data

The following **pidOutput()** function calculates the control signals needed to adjust the drone's motors based on the difference between the desired and actual states. It implements a PID (Proportional-Integral-Derivative) control algorithm to stabilize the drone by correcting errors in roll, pitch, yaw, and throttle.

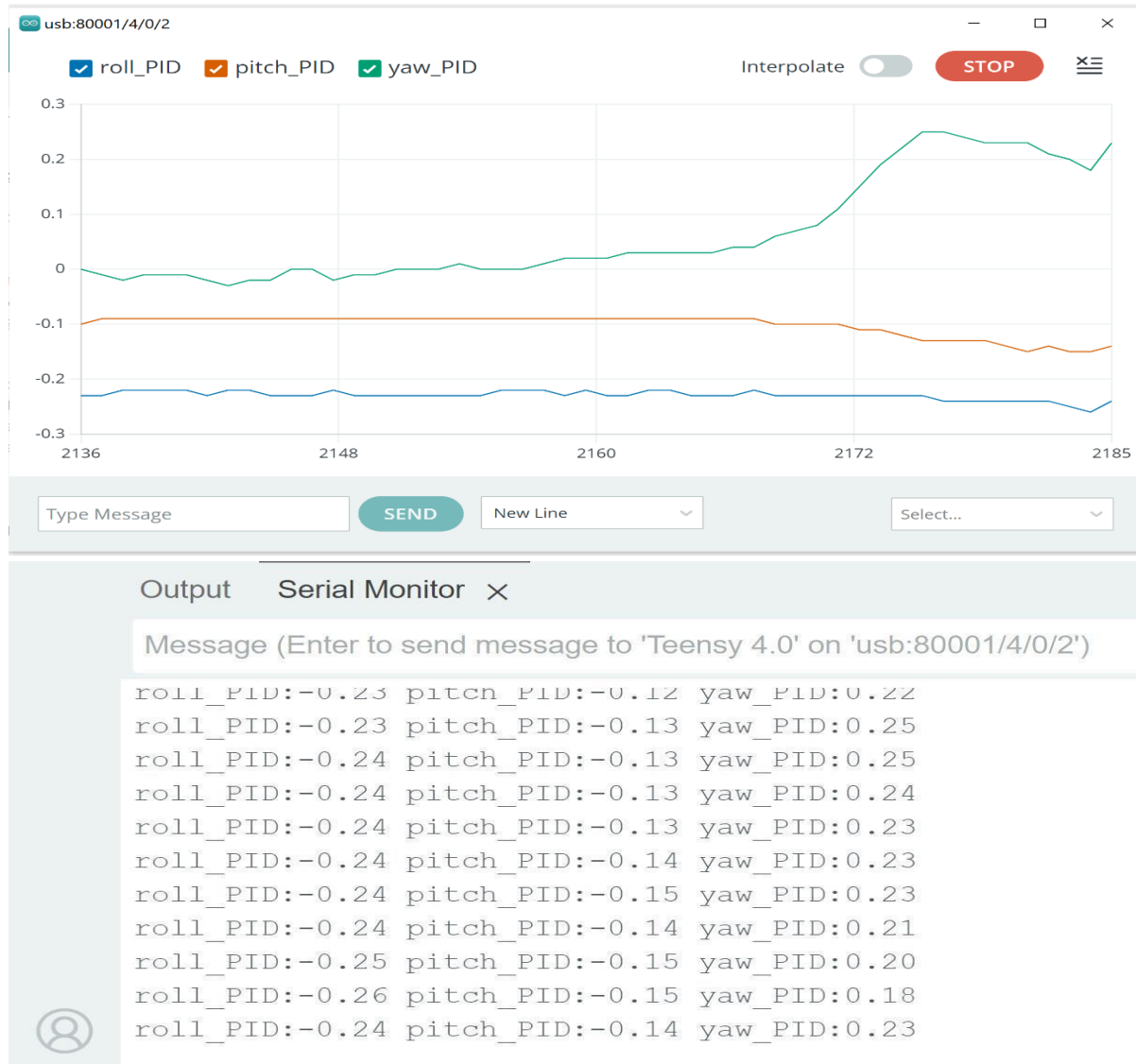


Fig 34: Roll pitch yaw PID data

We have successfully completed the development and testing of the drone, and the results have been highly satisfactory. During flight trials, the drone exhibited stable flight performance, efficient power distribution, and precise control responsiveness. Sensor data from the IMU, GPS, barometer, and compass were accurately captured and processed by the flight controller, ensuring reliable stabilization and autonomous navigation.

In addition to testing, we thoroughly verified sensor data values, implemented and fine-tuned the control algorithms, and achieved balanced and consistent outputs. Importantly, our custom-developed flight controller code has been designed with flexibility in mind—it performs reliably not only with standard multirotor configurations but also adapts seamlessly to more complex platforms such as VTOL (Vertical Take-Off and Landing) drones. This demonstrates the robustness, scalability, and versatility of our system across multiple UAV architectures. All onboard components—including ESCs, motors, power distribution, and communication modules—functioned flawlessly, confirming the effectiveness and reliability of the complete integration.



Fig 35: Assembled quadcopter frame with integrated electronics

CHAPTER 8

CONCLUSION

The culmination of this project lies in developing an **advanced flight controller and flight computer system** for UAVs, capable of **precise flight stabilization and autonomous decision-making**. Through the **PID controller**, smooth control of roll, pitch, and yaw is achieved by dynamically adjusting motor speeds to maintain stability even under disturbances. The **flight computer integrates data from multiple sensors**, relying on **sensor fusion algorithms** like Complementary, Madgwick, and Kalman filters to accurately estimate the UAV's orientation and minimize errors from noisy measurements.

This seamless control system enables the UAV to **perform complex autonomous operations**, including **obstacle avoidance, human tracking, and autonomous missions**. With obstacle sensing and course corrections, the UAV navigates challenging environments efficiently. The **integration of flight dynamics with real-time sensor feedback** ensures stable flight while performing tasks such as following moving targets or autonomously navigating predefined paths.

This project's final result is a **highly responsive and adaptive UAV control system**—one that demonstrates robust performance across varied scenarios. The system ensures safe operation and reliability, opening avenues for both commercial and research applications, from search-and-rescue missions to environmental monitoring. Through **fine-tuned control algorithms and real-time sensor fusion**, this UAV platform offers enhanced functionality and sets a foundation for **future innovations in drone technology**.

CHAPTER 9

Future Prospects for the UAV Flight Control System

The completion of the flight control system marks a significant milestone in the development of an advanced UAV platform. Building on this foundation, the future phases of the project will focus on enhancing the UAV's capabilities to achieve full autonomy and expand its potential applications. Below are the future prospects for the system:

1. Development of an Intelligent Flight Computer

With the flight control system operational, the next critical step is to design and implement a sophisticated flight computer. This component will process data from multiple onboard sensors and enable advanced functionalities such as real-time navigation, obstacle avoidance, and mission planning. Key areas include:

- **Sensor Integration:** Combining data from LiDAR, cameras, GPS, and IMU to achieve comprehensive situational awareness.
 - **Sensor Fusion:** Employing techniques like Extended Kalman Filters (EKF) or complementary filters to merge sensor inputs, ensuring accurate orientation, positioning, and environmental perception.
 - **Processing Algorithms:** Implementing robust algorithms for analyzing sensor data to make informed decisions about navigation and obstacle detection.
-

2. Autonomous Navigation and Obstacle Avoidance

To achieve true autonomy, the UAV must navigate dynamic environments independently. Future efforts will focus on:

- **Path Planning:** Developing algorithms to generate optimal flight paths, avoiding obstacles and ensuring efficient navigation.
 - **Real-Time Obstacle Detection:** Utilizing LiDAR, ultrasonic sensors, or stereo cameras to identify and avoid obstacles during flight.
 - **Dynamic Adaptation:** Enabling the UAV to respond to changes in its environment, such as moving objects or varying weather conditions.
-

3. Human Tracking and Interaction

Human tracking will play a vital role in applications like search and rescue, surveillance, and delivery services. Enhancements in this area include:

- **Target Identification:** Using computer vision techniques to identify and track human subjects accurately.
 - **Behavior Prediction:** Incorporating AI-based models to predict human movement patterns and optimize UAV response.
 - **Human Interaction:** Developing safe and effective methods for UAVs to interact with humans, such as delivering payloads or providing visual feedback.
-

4. Mission Planning and Execution

Advanced mission capabilities will enable the UAV to perform complex tasks with minimal human intervention. Key goals are:

- **Pre-Defined Missions:** Creating software for designing and executing predefined missions, such as mapping or inspection.
 - **Dynamic Mission Updates:** Allowing mid-flight mission adjustments based on environmental data or operator input.
 - **Multi-Agent Coordination:** Exploring swarm robotics to coordinate multiple UAVs for tasks like large-scale mapping or disaster response.
-

5. Integration of AI and Machine Learning

Integrating AI and machine learning technologies will unlock new capabilities for the UAV:

- **Autonomous Decision-Making:** Training models to make high-level decisions based on sensor data.
 - **Fault Detection and Recovery:** Using AI to identify system anomalies and execute recovery procedures.
 - **Environment Learning:** Allowing the UAV to learn from its surroundings and improve performance over time.
-

6. Expanding Applications

The advancements in the flight control system and flight computer will pave the way for UAV applications across various industries:

- **Search and Rescue:** Enhanced human tracking and obstacle navigation to locate and assist individuals in emergencies.
- **Inspection and Monitoring:** Using autonomous navigation for infrastructure inspection or environmental monitoring.
- **Logistics and Delivery:** Efficient route planning and obstacle avoidance for package delivery.

CHAPTER 10

REFERENCE

- [1] Mur-Artal, R., Montiel, J. M. M., & Tardós, J. D. (2015). ORB-SLAM: A Versatile and Accurate Monocular SLAM System. *IEEE Transactions on Robotics*.
- [2] Loquercio, A., Kaufmann, E., Ranftl, R., Müller, M., Koltun, V., & Scaramuzza, D. (2018). Deep Drone Racing: From Simulation to Reality with Domain Randomization. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- [3] Yang, H., Lee, S., & Lee, J. (2019). Energy-Efficient Path Planning for UAVs in Wind Fields. *Journal of Guidance, Control, and Dynamics*.
- [4] Saska, M., Vakula, J., & Přeučil, L. (2016). Model Predictive Control for Swarms of UAVs. *Autonomous Robots*.
- [5] Alexis, K., Papachristos, C., Siegwart, R., & Tzes, A. (2017). Robust UAV Control Under Wind Disturbances. *IEEE Transactions on Control Systems Technology*.
- [6] Krajník, T., Vonásek, V., Fišer, D., & Faigl, J. (2017). Long-Term Vision-Based Target Tracking in UAVs. *Journal of Field Robotics*.
- [7] Mozaffari, M., Saad, W., Bennis, M., & Debbah, M. (2019). UAV-Enabled Wireless Networks: A Survey. *IEEE Communications Surveys & Tutorials*.
- [8] Graule, M., Chirarattananon, P., Fuller, S., Jafferis, N., Ma, K., Spenko, M., ... & Wood, R. (2016). Perching via Controlled In-Flight Charging. *Science Robotics*.
- [9] Kaufmann, E., Loquercio, A., Ranftl, R., Müller, M., Koltun, V., & Scaramuzza, D. (2023). Champion-Level Drone Racing with Reinforcement Learning. *Nature*.
- [10] Park, J., Kim, Y., & Lee, J. (2021). TDMA Protocols for Multi-UAV Networks. *IEEE Transactions on Vehicular Technology*.

- [11] Rathi, S., Agrawal, A., & Ghosh, A. K. (2020). CFD Analysis of Hybrid VTOL Transitions. *AIAA Journal*.
- [12] Vega, F., Ramírez, F., Saiz, M., & Rosúa, F. (2018). UAVs in Precision Agriculture. *Remote Sensing*.
- [13] Alladi, T., Chamola, V., Sikdar, B., & Choo, K. R. (2022). Blockchain for UAV Security. *IEEE Internet of Things Journal*.
- [14] Kopardekar, P., Rios, J., Prevot, T., Johnson, M., Jung, J., & Robinson, J. E. (2020). NASA's UTM: Scaling Drone Operations. *Journal of Air Transportation*.
- [15] Zhang, Y., Liu, X., & Chen, B. M. (2021). Fault-Tolerant PID Control for UAVs. *IEEE/ASME Transactions on Mechatronics*.
- [16] D. Mellinger and V. Kumar, "Minimum Snap Trajectory Generation and Control for Quadrotors," *2011 IEEE International Conference on Robotics and Automation*, Shanghai, China, 2011, pp. 2520-2525.
- [17] J. Kim, S. A. Gadsden, and S. A. Wilkerson, "A Comprehensive Survey of Control Strategies for Autonomous Quadrotors," *Journal of Control Science and Engineering*, vol. 2017, Article ID 4671086, 2017.
- [18] Zulu, and John, "Review of Control Strategies for Quadrotors," *International Journal of Robotics Research*, vol. 39, no. 2, pp. 123-134, 2020.
- [19] S. Bouabdallah, "Design and Control of Quadrotors for Autonomous Flight," Master's thesis, École Polytechnique Fédérale de Lausanne, Lausanne, Switzerland, 2006.
- [20] R. M. Shubham and G. K. Gupta, "Unmanned Aerial Vehicles (UAVs): Applications and Future Trends," *Journal of Aerospace Engineering*, vol. 232, no. 2, pp. 1-15, 2022.
- [21] S. N. K. Murthy and S. Kumar, "Control Strategies for Quadrotors: A Survey," *IEEE*

Transactions on Robotics, vol. 33, no. 3, pp. 529-543, 2017.

[22] C. W. Lee, "PID Control Implementation for Quadrotor UAVs," *Journal of Control Science and Engineering*, vol. 2016, Article ID 2368657, 2016.

[23] Rehm, Nicholas, "dRehmFlight VTOL Flight Controller" url =
{<https://github.com/nickrehm/dRehmFlight>} Jan 2020

